Break It Down, Pass It On: Cross-Task Skill Transfer in LLM Agents

Yiyang Feng Biddut Sarker Bijoy Niranjan Balasubramanian Jiawei Zhou
Computer Science Department, Stony Brook University, Stony Brook, USA
{yiyfeng, bbijoy, niranjan, jiawei.zhou.1}@cs.stonybrook.edu

Abstract

Large language model (LLM) agents can induce skills from completed tasks and reuse them later to grow more capable with experience. In practice, induced skills may transfer unreliably and can even harm the agent that retrieves them. When agent-induced skills transfer reliably across tasks remains an open question. We conduct a comprehensive and controlled study of how the way skills are induced shapes their transfer across tasks. Specifically, we compare task-level with subtask-level skill induction and text with code skill formats, the two axes along which existing methods differ. Task-level skills mostly reduce the agent's performance below its no-memory baseline while subtask-level skills raise it above on average, and text skills transfer better than code skills. To further understand our findings, we examine two complementary properties of the induced skills: *specificity*, which measures how closely a skill matches real tasks, and *abstractness*, which measures how evenly its relevance spreads across tasks. Neither property alone predicts task success, but their combined effect does, which we propose as a *skill utility* score. The score correlates consistently with task success when skills are transferred, and subtask-level and text skills score higher. Computing skill utility only needs the skills and task descriptions but not any task execution, so our score serves as a practical diagnostic of a skill memory before any new task runs.[1]

1 Introduction

Despite the wide use of large language model (LLM) agents in workflows such as personal assistance, office automation, and scientific research (Trivedi et al., 2024; Wang et al., 2024c; Lai et al., 2026), these agents in their naive form solve each task from scratch and do not get better with experience. A growing line of work enables agents to induce skills from completed tasks, store them in a skill memory, and retrieve them for later tasks (Wang et al., 2024a; Zhao et al., 2024; Wang et al., 2025c,b). If the induced skills transfer across tasks, an agent becomes increasingly capable over time.

Figure 1: Different tasks share subtasks. A task-level skill summarizes the whole trajectory, so it stays tied to its source task and is not shared with the other task. Subtask-level induction yields one skill per subtask, and the skills of shared subtasks (amber) transfer between Task X and Task Y.

In practice, however, skill transfer is unreliable when skills are induced from the whole trajectories of completed tasks. First, a skill induced from a whole trajectory is tied to its source task (Fig. 1) and generalizes poorly to new tasks (Yu et al., 2025; Fang et al., 2026). Second, these over-specialized skills enter later tasks as irrelevant or misaligned context, which distracts the model and propagates errors from the source task (Shi et al., 2023; Yoran et al., 2024; Xiong et al., 2026). Therefore, whether skill transfer helps depends on how skills are induced, raising our central question: *when do agent-induced skills transfer reliably across tasks?*

A recent line of work suggests inducing skills at the subtask level (Nottingham et al., 2024; Yang et al., 2026; Shen et al., 2026). Given that tasks often share sub-procedures, agents could induce one

[1]Code and data are available at https://github.com/Zesearch/skill-transfer-llm-agents.

skill from each decomposed subtask rather than from the whole trajectory (Fig. 1), and early results report better performance. Yet the evidence for this gain stays narrow, covering limited domains, few models, and text skills only. Answering our central question therefore requires an analysis that compares task-level with subtask-level induction under identical conditions across various skill formats, domains, and models.

We conduct a comprehensive and controlled analysis of how agents should induce skills for reliable cross-task transfer. Our analysis varies two controls, the skill induction level, whether a skill summarizes a whole trajectory or a single subtask, and the skill format, whether a skill is stored as text or as code. Our experiments cover three long-horizon benchmarks and 11 open-weight and proprietary models. The skill induction level decides whether skills help. Subtask-level skills lift the agent above the no-memory baseline on average, while task-level skills tend to harm the same agent. The skill format then decides how much skills help or hurt, as text skills transfer better than code skills at both skill induction levels (Sec. 5).

To further understand these findings, we examine two complementary properties of the induced skills: *specificity*, which measures how closely a skill matches real tasks, and *abstractness*, which measures how evenly its relevance spreads across tasks. We show that neither property alone predicts task success, but their product does. We define this product as a *skill utility* score. Consistent with our findings, subtask-level and text skills have higher skill utility (Sec. 6).

Our analysis answers the opening question with three takeaways for practitioners. First, decomposing a task into subtasks and inducing one skill per subtask improves cross-task skill transfer. Second, text skills transfer better than code skills at both levels. Finally, skill utility serves as a lightweight diagnostic tool, so a practitioner can assess a skill memory before any new task runs.

2 Related Work

Skills in agents. A line of work consolidates an agent's experience into an explicit skill memory (Sharma et al., 2022; Park et al., 2023), differing along three axes (Tab. 1): (*i*) the skill induction level, including a trajectory (Wang et al., 2024a; Zhao et al., 2024; Shinn et al., 2023; Majumder et al., 2024; Wang et al., 2025c), a step (Nguyen et al., 2025; Wang et al., 2026), a rule (Chen et al., 2024; Fu et al., 2024), or a subtask (Nottingham et al., 2024; Yang et al., 2026; Shen et al., 2026); (*ii*) the skill format, including text (Zhao et al., 2024; Zhu et al., 2023; Majumder et al., 2024; Fang et al., 2026) or code (Wang et al., 2024b, 2025b; Ellis et al., 2021; Grand et al., 2024; Cai et al., 2024; Qian et al., 2023; Yuan et al., 2024); and (*iii*) the induction prompt, with various instructions, demonstrations, and abstraction rules (Hu et al., 2026). Others train memory operations or skill-augmented policies with RL (Yan et al., 2026; Xia et al., 2026; Feng et al., 2026a; Li et al., 2026) or retrieve episodes without induction (Zheng et al., 2024; Zhang et al., 2026; Ahmed et al., 2026; Yang et al., 2024). We vary the level and the format under one shared prompt, isolating each axis.

Cross-Task Skill Transfer. Existing task-level skill transfer shows gains across multiple domains, covering the web (Wang et al., 2025c,b; Zheng et al., 2025; Zhou et al., 2025b), GUI control (Wang et al., 2025a), embodied worlds (Sarch et al., 2024), software (Ouyang et al., 2026) or research agents (Zhou et al., 2025a). Despite the breadth of work on task-level skills, the study of subtask-level skill transfer stays limited. (*i*) Each work evaluates its skills on limited domains (Nottingham et al., 2024; Yang et al., 2026; Shen et al., 2026). (*ii*) End-to-end scores hide each skill's contribution to success despite using statistics including reuse counts or library growth (Zhong et al., 2026; Tan et al., 2025; Fang et al., 2026; Yu et al., 2025). (*iii*) Most work shows explicitly that skills bring gains, but other work implies the opposite, as irrelevant context degrades models (Shi et al., 2023; Yoran et al., 2024; Cuconasu et al., 2024; Liu et al., 2024), misaligned memories propagate errors (Xiong et al., 2026; Feng et al., 2025), and even relevant memories can fail to propagate (Zhong et al., 2023; Feng et al., 2026b). We therefore study when skill transfer helps or hurts, isolating each choice and validating a per-skill utility score against success.

Subtask decomposition in long-horizon agents. Prior work breaks a complex question into atomic sub-questions that are easier to solve (Zhou et al., 2023; Khot et al., 2023; Dua et al., 2022; Prasad et al., 2024; Sun et al., 2023). Long-horizon agents adopt the same idea to keep the growing context short and tackle each subtask in long-horizon tasks (Hu et al., 2025; Ye et al., 2026; Sun et al., 2026). However, the subtask boundary serves only the current task, and no skill transfers across tasks.

Work	Skill Induction Level	Skill Format	Domain	Instruction	Demonstration	Abstraction Rules
AutoManual (Chen et al., 2024)	Rule	Text	Household, Websites	✓	✓	✓
DynaSaur (Nguyen et al., 2025)	Step	Code	Assistance, Math, QA	✓	✓	✓
Reflexion (Shinn et al., 2023)	Task	Text	QA, Household, Code	✓	✓	×
ExpeL (Zhao et al., 2024)	Task	Text	QA, Household, Websites	✓	×	✓
CLIN (Majumder et al., 2024)	Task	Text	Science Simulation, Household	✓	×	✓
AWM (Wang et al., 2025c)	Task	Text	Websites	✓	✓	✓
Memp (Fang et al., 2026)	Task	Text	Travel, Household	✓	✓	×
Voyager (Wang et al., 2024a)	Task	Code	Game	✓	✓	✓
TroVE (Wang et al., 2024b)	Task	Code	Math, Table, Vision	✓	✓	×
ASI (Wang et al., 2025b)	Task	Code	Websites	✓	✓	✓
SSO (Nottingham et al., 2024)	Subtask	Text	Science Simulation, Game	✓	×	✓
MUSE (Yang et al., 2026)	Subtask	Text	Productivity	✓	×	✓
Shen et al. (2026)	Subtask	Text	Software Engineering	n/r	n/r	n/r
Ours	Task, Subtask	Text, Code	Assistance, Office, Data Science	✓	✓	✓

Table 1: Survey of skill-induction methods. Skill induction level is the span a skill is induced over, skill format is the representation of a stored skill, and domain lists the evaluation domains in each work's experiments. The last three columns mark whether the induction prompt includes an instruction that states what to extract, a demonstration that shows one worked extraction, and abstraction rules that drop instance-specific detail. The prompt cells are read from each system's released prompt files or paper appendix, and n/r means the prompt is not released.

Some works transfer subtask skills across tasks, but they are limited to one or two domains and few models (Nottingham et al., 2024; Yang et al., 2026; Shen et al., 2026). We instead study cross-task skill transfer at both induction levels, across two skill formats, three domains, and eleven models.

3 Cross-Task Skill Transfer

This section formalizes cross-task skill transfer, where an LLM agent solves a stream of tasks and its experience on earlier tasks affects how it solves later ones. We introduce two agents that map to two skill induction levels (Sec. 3.1), a skill memory that induces, stores, and retrieves skills in two different formats (Sec. 3.2), and a definition of cross-task skill transfer (Sec. 3.3). Fig. 2 shows the illustration, and the full prompts are in App. A.

3.1 Agents

An agent solves each task in a partially observable environment (Kaelbling et al., 1998). A task specifies a natural-language goal and an initial environment state. At step t, an LLM policy π generates an action a_t based on the interaction so far, and the environment returns an observation o_t. When the agent signals completion or reaches a step limit, an evaluator gives the final state a reward $r \in [0, 1]$.

Task-level agent. The task-level agent is a flat ReAct loop (Yao et al., 2023). It appends ($\oplus$) every action and observation into one context $h_t = h_{t-1} \oplus (a_t, o_t)$ that grows from an initial prompt h_0 encoding the goal, and draws each action a_t from $\pi(\cdot \mid h_{t-1})$. The loop ends in the whole-task trajectory $\tau = h_0 \oplus (a_1, o_1) \oplus \cdots \oplus (a_T, o_T)$.

Subtask-level agent. The subtask-level agent runs the same environment and policy, but it decomposes each task into subtasks (Zhou et al., 2023; Prasad et al., 2024) with a cycle of three roles. A planner reads the goal and the running summary, then proposes the next subtask or declares the task complete. An executor runs a ReAct loop on the current subtask, producing a sub-trajectory τ_k for subtask k. A summarizer then compresses τ_k into the running summary for the next cycle (Hu et al., 2025). The cycle repeats until the planner declares completion or a subtask limit is reached, ending in the task trajectory $\tau = (\tau_1, \ldots, \tau_K)$. The task-level agent is thus a special case of the subtask-level agent, where the whole task forms the single subtask and only the executor runs.

3.2 Skill Memory

A skill memory M stores skills from the agent's experience for reuse on later tasks. An induction operator turns the trajectory of a completed task or subtask into a skill, and a retrieval operator reads relevant skills from M before a task or subtask.

Skill format. Each skill has a short natural-language description (also used for retrieval), and a body that carries the main content in text or code format inspired by Wang et al. (2025c,b). A text skill writes the body as a workflow note, listing the procedure and its environment-specific caveats. A code skill writes the body as a Python function with instance-specific values as parameters and environmental caveats as code comments.

Skill induction. Skills are induced at two levels. Task-level induction turns the completed task trajectory τ into one skill, and subtask-level induction turns each completed sub-trajectory τ_k into one skill. This one-skill-per-trajectory rule prevents task-level induction from splitting a trajectory into several subtask-like skills, which would blur the

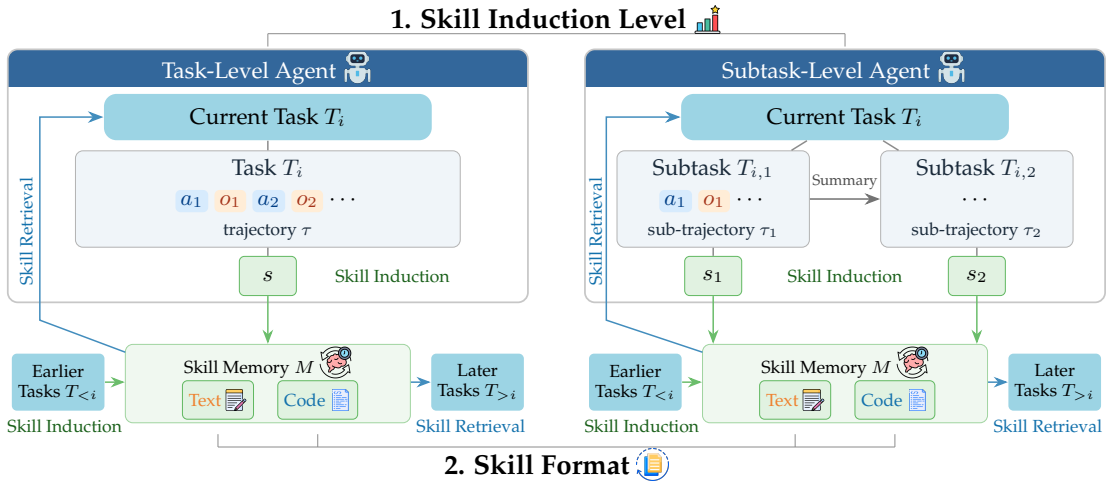

Figure 2: Illustration of two skill induction levels and two skill formats. (*i*) The task-level agent runs the current task as one trajectory of actions and observations and induces one skill from the whole trajectory. The subtask-level agent decomposes the task into subtasks, passes a running summary between consecutive subtasks, and induces one skill from each sub-trajectory. The two agents differ only in the skill induction level. (*ii*) Each agent stores its induced skills in its own memory as text notes or code functions and retrieves relevant skills back into its context, so skills induced on earlier tasks serve later ones. Crossing the two levels with the two formats and a no-memory baseline gives the six conditions we compare.

two levels. The two levels share the same induction prompt, so the contrast isolates only the skill induction level. Examples are in App. C.

Skill retrieval. The retrieval operator $R(M, q)$ embeds skill descriptions and the query q with an embedding model[2] and injects the top matches into the agent's context. Text and code skills are retrieved in the same way by matching the description only. The task-level agent queries once with the task instruction, and the subtask-level agent queries at each subtask with the task instruction or the subtask text. Every retrieved skill enters the context as its description and body, and a code skill is also loaded into the namespace so the agent can call it by name. The memory also prunes code skills that fail to load and merges or drops near-duplicate descriptions (App. B). We verify in App. E.6 that the differences among skill conditions come from the skills themselves rather than from retrieval quality.

3.3 Cross-Task Skill Transfer

An agent from Sec. 3.1 solves a stream of tasks $T_1, \ldots, T_n$ against the shared memory of Sec. 3.2. It writes one skill after each task or subtask and reads them before each, so a skill induced on an earlier task can be retrieved on a later one. We call such skill reuse *cross-task skill transfer*.

Design choices. Our study varies two choices. (*i*) The skill induction level is the span of task trajectory that each skill is induced from, the whole task for the task-level agent or a single subtask for the subtask-level agent. (*ii*) The skill format is how a skill is stored, either a text note, a code function, or none when the memory is off. Note that the two agents differ only in the skill induction level, plus the subtask-level agent's added planner and summarizer prompts. Every other aspect of the prompts, memory, and retrieval is identical. The remaining difference cancels in our comparisons, as each agent with skills is measured against the same agent without skills, so the results reflect the effect of the skills induced at each level rather than the agents themselves.

4 Experiment Setup

Benchmarks. We use three long-horizon benchmarks that span diverse domains. AppWorld (Trivedi et al., 2024) covers multi-app tool use in a Python REPL over nine simulated apps with documented APIs. OfficeBench (Wang et al., 2024c) covers office-document workflows over apps such as Email, Word, Excel, and PDF. KramaBench (Lai et al., 2026) covers data-science pipelines over real files from six scientific domains. We evaluate on the AppWorld test-challenge split (417 tasks), 300 OfficeBench tasks, and the 92 deterministically graded KramaBench tasks, each scored in $[0, 1]$ by its benchmark's official evaluator.

Models. We evaluate three Mixture-of-Experts (MoE) models (Qwen3-235B-A22B (Yang et al.,

[2]We use `all-MiniLM-L6-v2`, https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2.

2025), GPT-OSS-120B (Agarwal et al., 2025), and Nemotron-Super-120B (Chandiramani et al., 2026)), dense models of different sizes (Qwen3 at 4B, 8B, 14B, 32B (Yang et al., 2025) and Gemma-3 at 4B, 12B, 27B (Team et al., 2025)), and one commercial model (Gemini-3.1-Pro[3]). We serve the MoE models on Amazon Bedrock, Gemini-3.1-Pro on Google Cloud, and each dense model with vLLM on one 96 GB NVIDIA GH200. We keep each provider's default sampling, truncate generated tokens at 8192 per call, and give every model the same prompts. We also rerun the experiments on the three MoE models and Qwen3-32B with two reduced induction prompts as an ablation, including a minimal instruction (L1) and the instruction plus one demonstration (L2), against the full prompt (L3). Full details are in App. A.

Comparison Conditions. We cross the two axes of Sec. 3, skill induction level (Task-level or Subtask-level) and skill format (None, Text, or Code), into six conditions: Task, Task+Text, Task+Code, Subtask, Subtask+Text, and Subtask+Code, where Task and Subtask alone carry no memory. The "+Text" tag adds natural-language workflow notes with procedures and environmental caveats. The "+Code" tag adds Python functions with instance-specific values as parameters and environment caveats as code comments. The induction prompt is identical across the two levels, so each contrast isolates one axis. We compare each agent with skills against the same agent without skills, so differences between the two agents cancel out and the comparison reflects only the effect of the skills induced at each level. We limit the task-level agent at 50 ReAct steps per task and the subtask-level agent at 15 subtasks under a shared 50-step ReAct executor budget.[4]

Evaluation. We evaluate along two axes, performance and efficiency. For performance, we report task success, the average score in $[0, 1]$ from each benchmark's official evaluator. For efficiency, we report latency, the wall-clock time per task, and dependency, a measure of the compute spent on the growing context (Zhou et al., 2026) (App. D).

5 Skill Induction Level and Format Impact Whether a Skill Helps

We now test how each of the two induction choices affects cross-task skill transfer, varying the skill induction level under both skill formats (Sec. 5.1) and the format at both levels (Sec. 5.2).

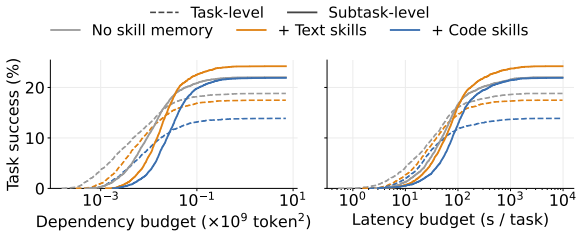

Figure 3: Task success rate within a per-task budget of dependency, i.e. the compute spent (left), and latency, i.e., the wall-clock time (right) per task (Sec. 4). We average over models and benchmarks for each skill format and skill induction level. The task-level agent wins only at the smallest budgets, and from moderate budgets on the subtask-level agent with skills solves more tasks at equal cost.

> **Takeaway.** Skills can help when they are induced at the subtask level, while task-level induction often makes the same memory harmful. Text skills mostly help more than code skills at both induction levels.

5.1 Skills Help at the Subtask Level

We first ask whether induced skills help the agents later, and how the answer depends on the skill induction level.

Performance (task success). Tab. 2 presents task success across the six conditions. For each task-level or subtask-level agent, we compare it with and without the induced memory as the effect of skills. Skills induced from whole tasks lower the task-level agent's average success, by 1.2 points with Text skills and 4.1 with Code, and on every benchmark, by up to 7.4 points. The same induction prompt applied at the subtask level instead raises average success, by 1.9 points with Text and 0.5 with Code. The subtask-level gain is consistent for Text skills, which help on all three benchmarks, while Code skills help on two of the three and lose 1.4 points on KramaBench, with individual models varying around these averages. To confirm that the effect travels with the skills rather than the agent, we equip the task-level agent with the subtask-level skills, which beat its own task-level skills on every benchmark (App. E.2). The effect is also not correlated by the outcomes of the source tasks, as the two levels induce from solved and unsolved tasks

[3] https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview

[4] The planner prompt requires an agent to generate at most 5 subtasks, which only serves as a soft control and the actual limit is 15.

Benchmark	Model	Task-Level			Subtask-Level		
		None	+Text	+Code	None	+Text	+Code
AppWorld	Qwen3-235B-A22B	7.4 [5.0, 10.1]	18.0 [14.4, 21.8]	14.4 [11.0, 18.0]	27.3 [23.0, 31.7]	35.5 [30.9, 40.0]	**35.7** [31.2, 40.3]
	GPT-OSS-120B	**27.3** [23.3, 31.7]	17.0 [13.4, 20.6]	1.0 [0.2, 1.9]	26.4 [22.3, 30.5]	25.2 [21.1, 29.5]	23.7 [19.7, 27.8]
	Nemotron-Super-120B	8.9 [6.2, 11.8]	4.3 [2.4, 6.2]	1.4 [0.5, 2.6]	17.7 [14.1, 21.6]	**21.6** [17.7, 25.4]	18.5 [14.9, 22.3]
	Qwen3-4B	0.0 [0.0, 0.0]	0.2 [0.0, 0.7]	0.0 [0.0, 0.0]	0.2 [0.0, 0.7]	0.7 [0.0, 1.7]	**1.4** [0.5, 2.6]
	Qwen3-8B	0.2 [0.0, 0.7]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	2.2 [1.0, 3.6]	**3.1** [1.7, 5.0]	1.9 [0.7, 3.4]
	Qwen3-14B	0.5 [0.0, 1.2]	0.7 [0.0, 1.7]	0.2 [0.0, 0.7]	6.7 [4.3, 9.4]	7.2 [4.8, 9.8]	**8.4** [5.8, 11.0]
	Qwen3-32B	1.9 [0.7, 3.4]	2.9 [1.4, 4.6]	1.4 [0.5, 2.6]	7.7 [5.3, 10.3]	**10.3** [7.4, 13.2]	7.2 [4.8, 9.8]
	Gemma-3-27B	0.5 [0.0, 1.2]	0.5 [0.0, 1.2]	0.0 [0.0, 0.0]	4.6 [2.6, 6.7]	4.3 [2.4, 6.5]	**4.8** [2.9, 7.0]
	Gemini-3.1-Pro	68.1 [63.5, 72.4]	58.0 [53.2, 62.6]	52.3 [47.5, 57.1]	68.3 [63.8, 72.7]	72.4 [68.1, 76.7]	**77.5** [73.4, 81.5]
	Average	10.4 [9.6, 11.3]	9.3 [8.4, 10.2]	6.5 [5.8, 7.1]	14.8 [13.5, 16.2]	**16.5** [15.1, 18.0]	16.4 [15.1, 17.7]
OfficeBench	Qwen3-235B-A22B	**43.0** [37.3, 48.7]	38.3 [33.0, 44.0]	36.7 [31.3, 42.0]	38.7 [33.3, 44.3]	41.3 [35.7, 47.0]	38.7 [33.3, 44.0]
	GPT-OSS-120B	32.3 [27.0, 37.7]	29.3 [24.3, 34.3]	5.0 [2.7, 7.7]	38.0 [32.7, 43.3]	**42.0** [36.7, 47.7]	40.7 [35.3, 46.3]
	Nemotron-Super-120B	28.3 [23.3, 33.7]	31.7 [26.3, 37.0]	12.7 [9.0, 16.7]	27.3 [22.3, 32.3]	**37.7** [32.3, 43.3]	25.0 [20.0, 30.0]
	Qwen3-4B	17.7 [13.3, 22.0]	16.3 [12.3, 20.7]	13.0 [9.3, 17.0]	18.0 [13.7, 22.7]	**25.3** [20.7, 30.3]	20.3 [16.0, 25.0]
	Qwen3-8B	**25.0** [20.0, 30.0]	15.7 [11.7, 20.0]	16.3 [12.3, 20.7]	19.0 [14.7, 23.7]	**25.0** [20.3, 30.0]	24.7 [19.7, 29.7]
	Qwen3-14B	28.3 [23.3, 33.7]	24.0 [19.3, 29.0]	27.3 [22.3, 32.3]	27.0 [22.0, 32.0]	**32.0** [27.0, 37.3]	30.3 [25.3, 35.7]
	Qwen3-32B	34.3 [29.0, 39.7]	32.7 [27.3, 38.0]	35.3 [30.0, 41.0]	33.3 [28.3, 38.7]	**36.0** [30.7, 41.7]	31.7 [26.7, 37.0]
	Gemma-3-27B	**28.3** [23.3, 33.7]	26.7 [21.7, 31.7]	14.7 [10.7, 18.7]	24.0 [19.0, 29.0]	19.0 [14.7, 23.7]	23.7 [19.0, 28.7]
	Gemini-3.1-Pro	46.3 [40.7, 52.0]	41.0 [35.3, 46.7]	41.7 [36.3, 47.3]	47.3 [41.7, 53.0]	46.0 [40.3, 51.7]	**48.7** [43.0, 54.3]
	Average	27.0 [23.6, 30.4]	25.3 [22.1, 28.8]	19.6 [17.0, 22.4]	26.3 [23.0, 29.7]	**29.7** [26.2, 33.2]	27.7 [24.2, 31.2]
KramaBench	Qwen3-235B-A22B	52.8 [43.0, 62.5]	49.3 [39.2, 59.2]	51.5 [41.1, 61.2]	52.4 [42.5, 62.2]	**55.3** [45.4, 64.7]	52.7 [42.9, 62.7]
	GPT-OSS-120B	31.7 [22.8, 41.1]	28.4 [19.6, 37.4]	22.5 [14.2, 31.1]	48.1 [38.3, 58.2]	49.8 [39.6, 59.5]	**50.9** [40.8, 60.6]
	Nemotron-Super-120B	53.3 [43.2, 63.2]	52.1 [42.1, 62.2]	43.3 [33.5, 53.4]	**59.8** [50.0, 69.1]	57.6 [47.6, 67.6]	49.9 [40.1, 59.7]
	Qwen3-4B	8.3 [3.3, 14.4]	12.3 [6.2, 19.0]	13.3 [6.9, 20.4]	13.8 [7.5, 20.8]	**15.1** [8.3, 22.5]	11.5 [5.5, 18.1]
	Qwen3-8B	10.7 [4.8, 17.2]	**15.5** [8.7, 22.9]	14.9 [8.3, 22.0]	14.4 [7.7, 21.5]	11.2 [5.4, 17.7]	14.1 [7.5, 21.5]
	Qwen3-14B	20.1 [12.5, 28.2]	14.9 [8.5, 22.0]	23.1 [14.7, 31.6]	19.1 [11.5, 27.1]	**23.5** [15.2, 32.5]	20.3 [12.5, 28.6]
	Qwen3-32B	30.0 [21.1, 39.2]	25.8 [17.6, 34.8]	30.2 [21.4, 39.3]	34.3 [25.0, 43.9]	**39.0** [29.3, 48.6]	38.9 [29.2, 48.7]
	Gemma-3-27B	19.7 [12.2, 27.7]	22.0 [14.0, 30.6]	18.6 [11.3, 26.5]	**25.3** [16.7, 34.4]	23.9 [15.9, 32.7]	24.2 [16.0, 33.1]
	Gemini-3.1-Pro	74.3 [65.6, 82.5]	74.1 [65.3, 82.4]	75.1 [66.4, 83.3]	**75.2** [66.5, 83.2]	73.7 [64.9, 82.0]	72.4 [63.6, 80.8]
	Average	29.0 [24.1, 34.0]	28.1 [23.2, 33.3]	27.8 [23.0, 32.9]	33.3 [28.0, 38.7]	**33.7** [28.5, 39.1]	31.9 [26.8, 37.1]
Average (11 models)		22.1 [20.2, 24.2]	20.9 [18.9, 22.9]	18.0 [16.1, 19.9]	24.8 [22.7, 27.0]	**26.7** [24.5, 28.8]	25.3 [23.3, 27.4]

Table 2: Task success (%) of the task-level and subtask-level agents, without (None) or with induced skills as text notes (+Text) or code functions (+Code), on AppWorld, OfficeBench, and KramaBench. We show nine of the eleven models and defer the two weakest, Gemma-3-4B and Gemma-3-12B, to Tab. 3 in App. E.1. Every Average row still covers all eleven models. Brackets are 95% task-bootstrap confidence intervals. On most models and benchmarks the highest success falls in a Subtask-level column, and adding skills mostly raises the success of the subtask-level agent while it lowers that of the task-level agent.

at close rates (App. E.3). Our conclusions also hold under various induction prompts (Fig. 26).

Efficiency (latency and dependency). We next examine task success under a per-task budget of latency and dependency, which measure the time and the compute spent on the growing context respectively (Sec. 4). Fig. 3 plots task success rate within each budget, so a vertical slice compares conditions at equal cost. Small budgets favor the task-level agent, which solves the easy tasks cheaply. From moderate budgets on, the subtask-level agent overtakes it under every format and saturates higher, so the subtask-level advantage is not bought by extra cost. The same crossover appears on every benchmark (Fig. 28). The within-agent contrast also survives cost matching. At equal cost the subtask-level agent with skills stays above its no-memory curve, while the task-level agent with skills stays below its own.

Varying task difficulty. The comparison of skill effect on task-level and subtask-level agents reaches the same conclusion at every task difficulty. We split tasks into easy, medium, and hard using each benchmark's official labels, including the annotated difficulty on AppWorld, the number of apps on OfficeBench, and the easy or hard tag on KramaBench. Fig. 4 compares all six conditions within each stratum. Task-level skills lower success in nearly every stratum under both formats, subtask-level skills raise it in most strata, and in nearly every stratum skills bring a larger gain at the subtask level than at the task level.

5.2 Text Skills Transfer Better than Code

Next, we vary the skill format under various skill induction levels.

Performance and efficiency. At the same skill induction level, retrieving text skills beats retrieving code skills, by 2.9 points on the subtask-level agent and 1.4 points on the task-level agent averaged over

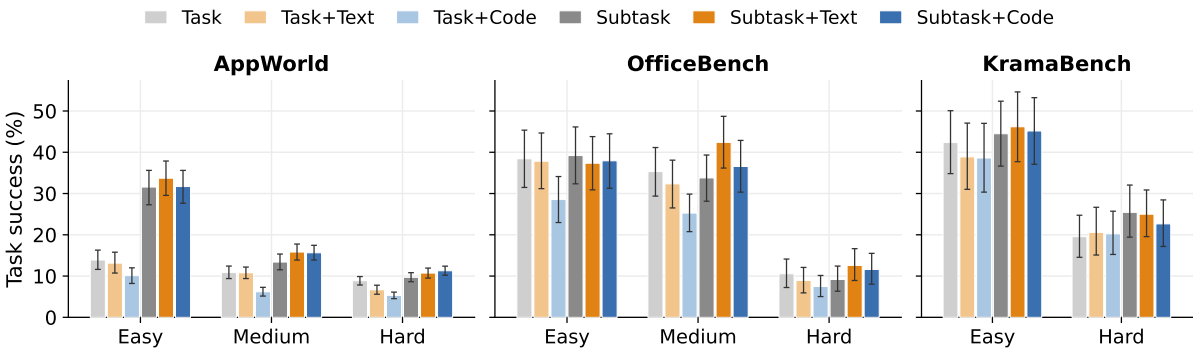

Figure 4: Task success of the six conditions by task difficulty, pooled over models with 95% task-bootstrap confidence intervals. In every stratum and under both formats, the subtask-level agent with skills stays above the task-level agent with skills. In nearly every stratum, the skills bring a larger gain to the subtask-level agent than to the task-level agent, and Text stays at or above Code within each induction level.

all models (Tab. 2). Relative to each agent's no-memory baseline, text skills damage the task-level agent less than code skills, 1.2 against 4.1 points, and help the subtask-level agent more, 1.9 against 0.5 points. The ranking also survives under various per-task efficiency budgets, as the Text curve stays at or above the Code curve within each induction level at every budget of dependency and latency (Fig. 3).

Varying task difficulty. Across all difficulty strata of Sec. 5.1, the Text conditions stay at or above the Code conditions within each skill induction level in nearly every stratum, so the format ranking also holds across difficulty (Fig. 4).

6 When Do Skills Transfer?

To understand when certain skills transfer better than others, we turn our focus from the agents to the skills they induce. We define a per-skill *utility score* computed from the induced skills and the task descriptions alone (Sec. 6.1), show that it explains why subtask-level and text skills transfer better (Sec. 6.2), and check that it agrees with the reuse observed during the task stream (Sec. 6.3).

Takeaway. A skill is useful when it is both specific to real tasks and abstract enough to remain relevant across many of them. Neither dimension alone predicts task success. We therefore propose a skill utility score that requires both jointly. Computed directly from the induced skills, it predicts task success and matches observed reuse.

6.1 A Score for Skill Utility

Existing literature hypothesizes a tension between a skill's specificity and its generalizability (Yu et al., 2025; Fang et al., 2026). We suggest that a reusable skill must meet two requirements, specificity and abstractness. Specificity asks the skill to be relevant enough to real tasks, and abstractness asks the skill to stay relevant to many tasks rather than a few. Suppose c_j denotes the cosine similarity between a skill s and the jth of the N task instructions under the retrieval embedder.

Specificity measures how close a skill is to the tasks it most resembles. Following Ethayarajh (2019), we compare the skill's nearest-task similarity with the similarities between tasks,

$$\text{specificity}(s) = \Pr\left[\max_j c_j \geq \cos(t_i, t_k)\right],$$

using the probability that the skill is closer to its nearest task than two random tasks t_i and t_k of the benchmark are to each other. Specificity punishes an irrelevant skill far from every task, which scores near zero, and rewards a skill matching at least one task, which scores near one.

Abstractness measures how evenly a skill's relevance spreads over many tasks set rather than concentrating on a few tasks. We turn the similarity c into a distribution and compute the perplexity over the task count to represent the ratio of relevant tasks to a skill (van der Maaten and Hinton, 2008),

$$\text{abstractness}(s) = \frac{\exp H\big(\text{softmax}(c/\tau)\big)}{N},$$

where $c = (c_1, \ldots, c_N)$, H is the entropy, and $\tau = 0.1$ is a temperature. Abstractness punishes a skill that is close to a few tasks but far from the

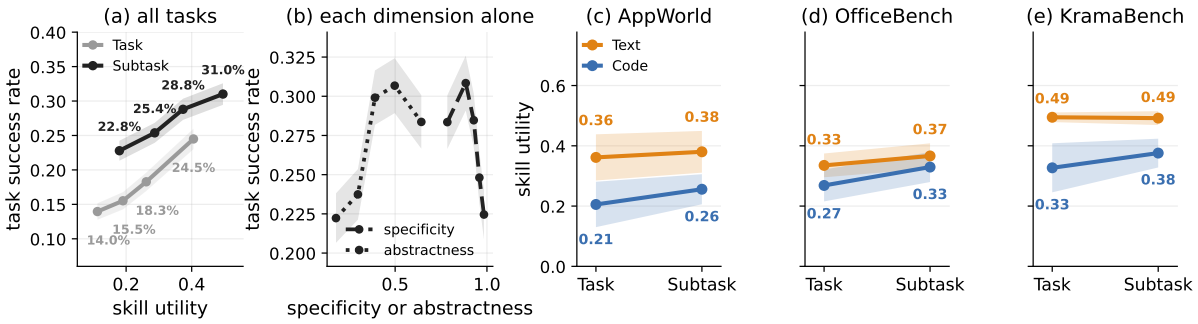

Figure 5: Skill utility against the results. (a): Each task is scored by the average utility of the agent-retrieved skills and ranked into equal-size bins per agent. Success rises from the lowest bin to the highest for both agents. (b): The subtask-level agent's tasks are scored by one dimension alone and split into equal-size bins, and success rises and then falls on both dimensions. (c)–(e): The median skill utility of each induced memory, averaged over models, is no lower at the subtask level than the task level, and higher for Text than Code. Shaded bands are 95% CIs.

rest, which scores near $1/N$, and rewards a skill that stays relevant across many tasks, which scores near one.

Skill utility. We show in Fig. 27 that specificity and abstractness trade off for both task-level and subtask-level skills. We hypothesize that a useful skill balances the two properties well, so we define skill utility as the product of both properties:

$$\text{utility}(s) = \text{specificity}(s) \cdot \text{abstractness}(s).$$

6.2 Skill Utility Predicts Task Success Gains

We examine whether skill utility explains accounts for the results of Sec. 5, that skills help mostly under subtask-level induction and that text skills transfer better than code skills.

Higher utility, more successes. If high-utility skills explain the task success gains, tasks that retrieve higher-utility skills should succeed more. We score each task by the average utility of the skills the agent retrieves when solving the task, rank tasks by the score for task-level and subtask-level agents, and split each ranking into equal-size bins. Success rises monotonically across the bins, from 14.0% to 24.5% for the task-level agent and from 22.8% to 31.0% for the subtask-level agent (Fig. 5a). Beyond the correlation between skill utility and task success, we also analyze the causal effect of skill utility. First, retrieved-skill utility stays nearly flat across each benchmark's native difficulty levels even though success falls steeply, and utility keeps predicting success within a fixed difficulty level. We also split the same skill library at its median utility and rerun the agent on the same tasks with one half at a time, and the high-utility half yields higher success for both the task-level and the subtask-level library (App. E.4).

Neither dimension, by itself, predicts success. Neither specificity nor abstractness alone predicts the success of the subtask-level agent. In Fig. 5b, success rises and then falls as specificity or abstractness increases. The phenomenon follows from the tradeoff in Fig. 27, as a skill high on one dimension gives up the other and loses utility. The agent therefore succeeds most when the two dimensions stay balanced, which only the product rewards.

Subtask-level and text skills have higher utility. We next compare the skill utility across the two induction levels and the two skill formats. For each condition we take the median utility of induced skills, which is robust to a few outliers, and average the medians over all models on each benchmark. From Fig. 5c to e, subtask-level skills score higher than task-level skills on almost every benchmark and skill format, with the only tie for text on KramaBench. Besides, text skills score higher than code at both induction levels on every benchmark.

6.3 Skill Utility Matches Actual Cross-Task Reuse

Finally, we check whether skill utility reflects the cross-task reuse that actually happens during the task stream.

Transfer density. We cut each benchmark's task stream into $n = 50$ equal bins in stream order. Transfer density is the share of ordered bin pairs that carry at least one actual transfer,

$$D = \frac{1}{\binom{n}{2}} \sum_{i<j} r_{ij},$$

where $r_{ij} = 1$ when a skill induced in bin i is retrieved in a later bin j, and 0 otherwise. We compute the density for each model and skill format and report the average over them.

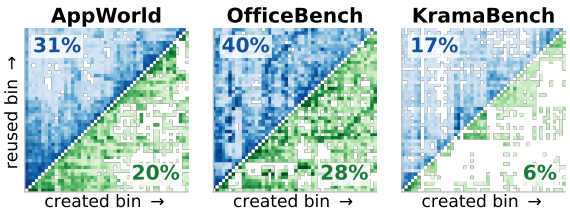

Figure 6: Transfer density. The task stream of each benchmark is cut into 50 bins in stream order, and a cell is colored when a skill induced in the earlier bin is retrieved in a later bin, darker when more of the runs do so. Blue upper triangles are the subtask-level agent and green lower triangles are the task-level agent, pooled over models and both skill formats, and the number in each triangle is its transfer density. The subtask-level agent reuses skills more densely than the task-level agent on all three benchmarks.

Results. The subtask-level agent reuses skills more densely than the task-level agent on all three benchmarks (Fig. 6). The skill induction level that scores higher utility is thus also the one whose skills are reused more often, so the score matches the transfer that actually happens. Each cell also shows where in the stream a skill was induced and reused.

7 Conclusion

We study when induced skills transfer across tasks, comparing different skill induction levels and skill formats on three benchmarks and eleven models. We show that subtask-level skills can improve the agent while task-level skills harm it, and text skills transfer better than code skills. Skill utility score, balancing specificity and abstractness, correlates with task success, ranks the winning conditions higher, and serves as an execution-free diagnostic of a skill memory before any task runs.

Limitations

Our study evaluates long-horizon agents on three standard benchmarks that together span multi-app tool use, office-document workflows, and data-science pipelines, each scored by its official evaluator. These choices give our conclusions a controlled basis, and extending them beyond this scope would need three additional studies. First, other agentic settings such as computer use (Xie et al., 2024), agentic coding (Deng et al., 2026; Merrill et al., 2026), and web search (Wei et al., 2025) may show different transfer behavior, so replicating our findings there requires additional experiments. Those environments require Docker with root access, and large-scale compute with that level of security access is difficult to obtain. Second, our skill memory follows fixed induction, retrieval, and deduplication rules, which keeps the six conditions comparable, while recent systems let the agent revise its stored skills and memory over time (Fang et al., 2026; Packer et al., 2024). Such revision needs a sandboxed file system, so studying it also faces the root Docker constraint noted above. Extending our findings to such evolving memories requires a separate study. Finally, we score each task by its final environment state, which keeps grading deterministic and comparable. The benchmarks provide no step-level ground truth, so a finer-grained analysis of intermediate decisions would need a judge model, and we leave it to future work.

Ethical Considerations

Our work probes when an LLM agent can reliably reuse skills induced from its own past experience. A skill memory that transfers procedures across tasks could also transfer harmful ones, because an adversary who injects malicious skills into the library could steer the agent through the same reuse mechanism. Our experiments carry no such risk, since every skill is induced by the agent itself from sandboxed benchmark tasks without real user data. We regard reliable reuse of trustworthy skills as a prerequisite for the harder challenge of deciding which stored skills to trust, so detecting and rejecting malicious skills is left for future work.

Acknowledgments

This research was supported by an Amazon Research Award of Spring 2025 on AWS Agentic AI, a Stony Brook Spring 2025 OVPR Seed Grant, and the National Artificial Intelligence Research Resource (NAIRR) Pilot program (award NAIRR250525). This research used the DeltaAI advanced computing and data resource, which is supported by the National Science Foundation (award OAC 2320345) and the State of Illinois. This work used Amazon Web Services, Google Cloud, and Microsoft Azure through the CloudBank project, which is supported by National Science Foundation grant #1925001. We also thank Huajian Zhang for feedback on the writing.

References

Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al.

2025. gpt-oss-120b & gpt-oss-20b model card. Preprint, arXiv:2508.10925.

Ammar Ahmed, Azal Ahmad Khan, Ayaan Ahmad, Sheng Di, Zirui Liu, and Ali Anwar. 2026. Retrieval-of-thought: Efficient reasoning via reusing thoughts. In The Fourteenth International Conference on Learning Representations.

Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2024. Large language models as tool makers. In The Twelfth International Conference on Learning Representations.

Aakshita Chandiramani, Aaron Blakeman, Abdullahi Olaoye, Abhibha Gupta, Abhilash Somasamudramath, Abhinav Khattar, Adeola Adesoba, Adi Renduchintala, Adil Asif, Aditya Agrawal, et al. 2026. Nemotron 3 super: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning. Preprint, arXiv:2604.12374.

Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. 2024. Automanual: Constructing instruction manuals by llm agents via interactive environmental learning. In Advances in Neural Information Processing Systems, volume 37, pages 589–631. Curran Associates, Inc.

Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonellotto, and Fabrizio Silvestri. 2024. The power of noise: Redefining retrieval for rag systems. In Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '24, page 719–729, New York, NY, USA. Association for Computing Machinery.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa R Kundurthy, Sean M. Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. 2026. SWE-bench pro: Can AI agents solve long-horizon software engineering tasks? In Forty-third International Conference on Machine Learning.

Dheeru Dua, Shivanshu Gupta, Sameer Singh, and Matt Gardner. 2022. Successive prompting for decomposing complex questions. In Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pages 1251–1265, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. 2021. Dreamcoder: bootstrapping inductive program synthesis with wake-sleep library learning. In Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2021, page 835–850, New York, NY, USA. Association for Computing Machinery.

Kawin Ethayarajh. 2019. How contextual are contextualized word representations? Comparing the geometry of BERT, ELMo, and GPT-2 embeddings. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 55–65, Hong Kong, China. Association for Computational Linguistics.

Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. 2026. Memp: Exploring agent procedural memory. In Findings of the Association for Computational Linguistics: ACL 2026, pages 17490–17502, San Diego, California, United States. Association for Computational Linguistics.

Xinshun Feng, Xinhao Song, Lijun Li, Gongshen Liu, and Jing Shao. 2026a. SEARL: Joint optimization of policy and tool graph memory for self-evolving agents. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 24518–24535, San Diego, California, United States. Association for Computational Linguistics.

Yiyang Feng, Zeming Chen, Haotian Wu, Jiawei Zhou, and Antoine Bosselut. 2026b. Tracking the limits of knowledge propagation: How LLMs fail at multi-step reasoning with conflicting knowledge. In Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pages 5813–5847, Rabat, Morocco. Association for Computational Linguistics.

Yiyang Feng, Yichen Wang, Shaobo Cui, Boi Faltings, Mina Lee, and Jiawei Zhou. 2025. Unraveling misinformation propagation in LLM reasoning. In Findings of the Association for Computational Linguistics: EMNLP 2025, pages 11683–11707, Suzhou, China. Association for Computational Linguistics.

Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. 2024. Autoguide: Automated generation and selection of context-aware guidelines for large language model agents. In The Thirty-eighth Annual Conference on Neural Information Processing Systems.

Gabriel Grand, Lionel Wong, Matthew Bowers, Theo X. Olausson, Muxin Liu, Joshua B. Tenenbaum, and Jacob Andreas. 2024. LILO: Learning interpretable libraries by compressing and documenting code. In The Twelfth International Conference on Learning Representations.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg,

Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, and 7 others. 2020. Array programming with NumPy. Nature, 585(7825):357–362.

Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. 2025. HiAgent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 32779–32798, Vienna, Austria. Association for Computational Linguistics.

Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, and 28 others. 2026. Memory in the age of ai agents. Preprint, arXiv:2512.13564.

J. D. Hunter. 2007. Matplotlib: A 2d graphics environment. Computing in Science & Engineering, 9(3):90–95.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. 1998. Planning and acting in partially observable stochastic domains. Artificial Intelligence, 101(1):99–134.

Anthony Kay. 2007. Tesseract: an open-source optical character recognition engine. Linux J., 2007(159):2.

Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2023. Decomposed prompting: A modular approach for solving complex tasks. In The Eleventh International Conference on Learning Representations.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23, page 611–626, New York, NY, USA. Association for Computing Machinery.

Eugenie Lai, Gerardo Vitagliano, Ziyu Zhang, Om Chabra, SIVAPRASAD SUDHIR, Anna Zeng, Anton A. Zabreyko, Chenning Li, Ferdi Kossmann, Jialin Ding, Jun Chen, Markos Markakis, Matthew Russo, Weiyang Wang, Ziniu Wu, Mike Cafarella, Lei Cao, Samuel Madden, and Tim Kraska. 2026. KRAMABENCH: A benchmark for AI systems on data-to-insight pipelines over data lakes. In The Fourteenth International Conference on Learning Representations.

Ruoran Li, Xinghua Zhang, Haiyang Yu, Shitong Duan, Xiang Li, Wenxin Xiang, Chonghua Liao, Xudong Guo, Yongbin Li, and Jinli Suo. 2026. MemPO: Self-memory policy optimization for long-horizon agents. In Findings of the Association for Computational Linguistics: ACL 2026, pages 23286–23301, San Diego, California, United States. Association for Computational Linguistics.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics, 12:157–173.

Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Peter Jansen, Oyvind Tafjord, Niket Tandon, Li Zhang, Chris Callison-Burch, and Peter Clark. 2024. CLIN: A continually learning language agent for rapid task adaptation and generalization. In First Conference on Language Modeling.

Mike A Merrill, Alexander Glenn Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, and 65 others. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In The Fourteenth International Conference on Learning Representations.

Dang Nguyen, Viet Dac Lai, Seunghyun Yoon, Ryan A. Rossi, Handong Zhao, Ruiyi Zhang, Puneet Mathur, Nedim Lipka, Yu Wang, Trung Bui, Franck Dernoncourt, and Tianyi Zhou. 2025. Dynasaur: Large language agents beyond predefined actions. In Second Conference on Language Modeling.

Kolby Nottingham, Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Sameer Singh, Peter Clark, and Roy Fox. 2024. Skill set optimization: Reinforcing language model behavior via transferable skills. In Forty-first International Conference on Machine Learning.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. Reasoningbank: Scaling agent self-evolving with reasoning memory. In The Fourteenth International Conference on Learning Representations.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. Memgpt: Towards llms as operating systems. Preprint, arXiv:2310.08560.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In Proceedings of the 36th Annual ACM Symposium on User Interface

Software and Technology, UIST '23, New York, NY, USA. Association for Computing Machinery.

Archiki Prasad, Alexander Koller, Mareike Hartmann, Peter Clark, Ashish Sabharwal, Mohit Bansal, and Tushar Khot. 2024. ADaPT: As-needed decomposition and planning with language models. In Findings of the Association for Computational Linguistics: NAACL 2024, pages 4226–4252, Mexico City, Mexico. Association for Computational Linguistics.

Cheng Qian, Chi Han, Yi Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models. In Findings of the Association for Computational Linguistics: EMNLP 2023, pages 6922–6939, Singapore. Association for Computational Linguistics.

Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 3982–3992, Hong Kong, China. Association for Computational Linguistics.

Gabriel Herbert Sarch, Lawrence Jang, Michael J. Tarr, William W. Cohen, Kenneth Marino, and Katerina Fragkiadaki. 2024. VLM agents generate their own memories: Distilling experience into embodied programs of thought. In The Thirty-eighth Annual Conference on Neural Information Processing Systems.

Pratyusha Sharma, Antonio Torralba, and Jacob Andreas. 2022. Skill induction and planning with latent language. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1713–1726, Dublin, Ireland. Association for Computational Linguistics.

Kangning Shen, Jingyuan Zhang, Chenxi Sun, Wencong Zeng, and Yang Yue. 2026. Structurally aligned subtask-level memory for software engineering agents. Preprint, arXiv:2602.21611.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. 2023. Large language models can be easily distracted by irrelevant context. In Proceedings of the 40th International Conference on Machine Learning, volume 202 of Proceedings of Machine Learning Research, pages 31210–31227. PMLR.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In Thirty-seventh Conference on Neural Information Processing Systems.

Haotian Sun, Yuchen Zhuang, Lingkai Kong, Bo Dai, and Chao Zhang. 2023. Adaplanner: Adaptive planning from feedback with language models. In Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023.

Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. 2026. Scaling long-horizon agent via context folding. In Forty-third International Conference on Machine Learning.

Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025. MemBench: Towards more comprehensive evaluation on the memory of LLM-based agents. In Findings of the Association for Computational Linguistics: ACL 2025, pages 19336–19352, Vienna, Austria. Association for Computational Linguistics.

Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. 2025. Gemma 3 technical report. Preprint, arXiv:2503.19786.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 16022–16076, Bangkok, Thailand. Association for Computational Linguistics.

Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-sne. Journal of Machine Learning Research, 9(86):2579–2605.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, and 16 others. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. Nature Methods, 17:261–272.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024a. Voyager: An open-ended embodied agent with large language models. Transactions on Machine Learning Research.

Jiongxiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. 2026. Reinforcement learning for self-improving agent with skill library. Preprint, arXiv:2512.17102.

Zhenhailong Wang, Haiyang Xu, Junyang Wang, Xi Zhang, Ming Yan, Ji Zhang, Fei Huang, and Heng Ji. 2025a. Mobile-agent-e: Self-evolving mobile assistant for complex tasks. In Workshop on Scaling Environments for Agents.

Zhiruo Wang, Graham Neubig, and Daniel Fried. 2024b. TroVE: Inducing verifiable and efficient toolboxes for solving programmatic tasks. In Forty-first International Conference on Machine Learning.

Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. 2024c. Officebench: Benchmarking language agents across multiple applications for office automation. Preprint, arXiv:2407.19056.

Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. 2025b. Inducing programmatic skills for agentic tasks. In Second Conference on Language Modeling.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025c. Agent workflow memory. In Forty-second International Conference on Machine Learning.

Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. 2025. Browsecomp: A simple yet challenging benchmark for browsing agents. Preprint, arXiv:2504.12516.

Wes McKinney. 2010. Data Structures for Statistical Computing in Python. In Proceedings of the 9th Python in Science Conference, pages 56 – 61.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. SkillRL: Evolving agents via recursive skill-augmented reinforcement learning. In ICLR 2026 Workshop on Lifelong Agents: Learning, Aligning, Evolving.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track.

Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. 2026. How memory management impacts LLM agents: An empirical study of experience-following behavior. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 623–645, San Diego, California, United States. Association for Computational Linguistics.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schuetze, Volker Tresp, and Yunpu Ma. 2026. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 12805–12825, San Diego, California, United States. Association for Computational Linguistics.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. Preprint, arXiv:2505.09388.

Cheng Yang, Xuemeng Yang, Licheng Wen, Daocheng Fu, Jianbiao Mei, Rong Wu, Pinlong Cai, Yufan Shen, Nianchen Deng, Jia Xu, Botian Shi, Yu Qiao, and Haifeng Li. 2026. Towards self-evolving agents: Enabling autonomy through interactive experience refinement. In Findings of the Association for Computational Linguistics: ACL 2026, pages 30424–30451, San Diego, California, United States. Association for Computational Linguistics.

Ling Yang, Zhaochen Yu, Tianjun Zhang, Shiyi Cao, Minkai Xu, Wentao Zhang, Joseph E. Gonzalez, and Bin CUI. 2024. Buffer of thoughts: Thought-augmented reasoning with large language models. In The Thirty-eighth Annual Conference on Neural Information Processing Systems.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In The Eleventh International Conference on Learning Representations.

Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, Pengjun Xie, Fei Huang, Jingren Zhou, Siheng Chen, and Yong Jiang. 2026. Agentfold: Long-horizon web agents with proactive context folding. In The Fourteenth International Conference on Learning Representations.

Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. 2024. Making retrieval-augmented language models robust to irrelevant context. In The Twelfth International Conference on Learning Representations.

Simon Yu, Gang Li, Weiyan Shi, and Peng Qi. 2025. Polyskill: Learning generalizable skills through polymorphic abstraction. Preprint, arXiv:2510.15863.

Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi Fung, Hao Peng, and Heng Ji. 2024. CRAFT: Customizing LLMs by creating and retrieving from specialized toolsets. In The Twelfth International Conference on Learning Representations.

Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. 2026. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. Preprint, arXiv:2601.03192.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: Llm agents are experiential learners. Proceedings of the AAAI Conference on Artificial Intelligence, 38(17):19632–19642.

Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. 2025. Skillweaver: Web agents can self-improve by discovering and honing skills. Preprint, arXiv:2504.07079.

Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. 2024. Synapse: Trajectory-as-exemplar prompting with memory for computer control. In The Twelfth International Conference on Learning Representations.

Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. 2026. Skilllearnbench: Benchmarking continual learning methods for agent skill generation on real-world tasks. Preprint, arXiv:2604.20087.

Zexuan Zhong, Zhengxuan Wu, Christopher Manning, Christopher Potts, and Danqi Chen. 2023. MQuAKE: Assessing knowledge editing in language models via multi-hop questions. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pages 15686–15702, Singapore. Association for Computational Linguistics.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V Le, and Ed H. Chi. 2023. Least-to-most prompting enables complex reasoning in large language models. In The Eleventh International Conference on Learning Representations.

Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, and Jun Wang. 2025a. Memento: Fine-tuning llm agents without fine-tuning llms. Preprint, arXiv:2508.16153.

Yifei Zhou, Qianlan Yang, Kaixiang Lin, Min Bai, Xiong Zhou, Yu-Xiong Wang, Sergey Levine, and Li Erran Li. 2025b. Proposer-agent-evaluator (PAE): Autonomous skill discovery for foundation model internet agents. In Forty-second International Conference on Machine Learning.

Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Bryan Kian Hsiang Low, and Paul Pu Liang. 2026. MEM1: Learning to synergize memory and reasoning for efficient long-horizon agents. In The Fourteenth International Conference on Learning Representations.

Xizhou Zhu, Yuntao Chen, Hao Tian, Chenxin Tao, Weijie Su, Chenyu Yang, Gao Huang, Bin Li, Lewei Lu, Xiaogang Wang, Yu Qiao, Zhaoxiang Zhang, and Jifeng Dai. 2023. Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory. Preprint, arXiv:2305.17144.

Table of Contents

A Prompts

We list the deployed prompts of the two agents and the two skill formats on the three benchmarks. Blue boxes hold system messages and yellow boxes hold user messages. Curly braces mark values filled at run time, and a gray hook marks a soft line wrap. The two induction levels share the skill induction prompt, and task-level agent is a special case of subtask-level agent because it has the whole task as the only one subgoal, so an induction level differs only in the trajectory span the induction step reads.

A.1 Subtask-Level Agent

The subtask-level agent runs a planner, an executor, and a summarizer, and the executor is the role that acts on the environment. The executor's system prompt is the execution core of Figs. 7, 8, and 9, followed by its role block in Figs. 10, 11, and 12.

The executor's user message (Fig. 13) carries the current subgoal, the task for context, the progress summary, and on OfficeBench the live action menu. The skills retrieved for a subgoal enter this message wrapped in the format framing of Fig. 14, so the executor treats remembered procedures as hints for the current subgoal only.

The planner and the summarizer reuse the same execution core and append their own role blocks. The planner is rebuilt fresh at every call, so it sees only the task and the current progress summary. The summarizer reads the executor conversation and replies with one progress summary tool call. Their user messages are also in Fig. 13.

A.2 Task-Level Agent

The task-level agent is the executor run as a special case whose single subgoal is the whole task. The planner and summarizer blocks drop away, and it has the same system and user prompts as the executor of the subtask agent. The only difference is that the single subgoal is the whole task instead of one produced by the planner. With skill memory on, the retrieved skills enter as one further user message under the framing of Fig. 14. A retrieved code skill is also loaded into the execution namespace so the agent can call the function by name.

A.3 Text-Skill Induction

After a task at the task level, or after each subgoal at the subtask level, the induction prompt of Figs. 15, 16, or 17 is appended to the finished span as a user message. The model must answer with one skill induction tool call whose arguments hold the description and the note body.

A.4 Code-Skill Induction

Code-skill induction follows the same protocol with the prompts of Figs. 18, 19, and 20. The tool call returns a functions list whose single entry holds a name, a description, and a Python implementation, and an empty list is reserved for spans with no real work.

B Skill Memory Mechanics

Both memories embed skill descriptions and queries with `all-MiniLM-L6-v2` and rank candidates by cosine similarity. Retrieval returns the top 5 text or code skills above 0.30. A selected code skill also pulls in the stored functions it calls. A new text skill merges into a stored entry whose description similarity exceeds 0.85, replacing the procedure line and appending only unseen bullets, and otherwise becomes a new entry. A new code skill overwrites the entry whose function name it reuses, is dropped as a near duplicate at description similarity 0.85, and otherwise becomes a new entry.

Execution feedback prunes code skills further. A stored function that fails to load into the execution namespace is removed. Each removal cascades to the functions whose source calls the removed one.

C Examples of Induced Skills

We show real skills induced by Qwen3-235B-A22B from one AppWorld task, which asks the agent to order all weightlifting benches in the Amazon cart. Each skill appears as induced right after its source trajectory, before any later merge. Task-level induction yields one skill for the whole task (Figs. 21 and 22), and subtask-level induction yields one skill per subtask, three in this run (Figs. 23 and 24). The scope contrast is easiest to see in the code format, as the task-level function keeps the source task's product keyword as its default argument while the subtask-level functions stay generic over login, cart filtering, and order placement.

D Evaluation Metrics

D.1 Performance Scoring

Each benchmark's official evaluator maps a finished task to a score in $[0, 1]$, and the reported task success of a condition is the mean score over its tasks. (*i*) AppWorld scores a task with its official state-based unit tests, which check the final environment state left by the agent's API calls. The score is 1 when every test passes and 0 otherwise. (*ii*) OfficeBench attaches official checker functions to each task, which inspect the files and application state the agent leaves in the sandbox. The score is 1 when every checker passes and 0 otherwise. (*iii*) KramaBench grades the submitted answer against the gold answer with the official deterministic scorer of its answer type, and a task that never submits scores 0. Exact numeric and string answers score 0 or 1 under a small numeric tolerance. List answers score the F1 overlap with the gold list, and approximate numeric answers score $1/(1+e)$ for a relative absolute error e. The per-task score is therefore continuous, and the few tasks graded by an LLM judge are excluded from our task set.

D.2 Cost Metrics

The latency of a task is its wall-clock time. The dependency is the MEM1 cost (Zhou et al., 2026), which approximates the attention work a task spends on its context. For a task whose model calls each have an input length p_c and an output length o_c, it is

$$\text{dependency} = \sum_c \frac{(2p_c + o_c)\, o_c}{2},$$

reported in units of 10^9 token^2. Both costs are read from the execution logs and never steer the agents.

System

```
You are an AI Assistant that completes tasks autonomously by interacting with apps through their APIs in a Python REPL
↪environment.

<APPWORLD_APIS>
Key APIs to discover available functionality:
```python
# To get a list of apps that are available to you.
print(apis.api_docs.show_app_descriptions())

# To get the list of apis under any app listed above, e.g. supervisor
print(apis.api_docs.show_api_descriptions(app_name='supervisor'))

# To get the specification of a particular api, e.g. supervisor app's show_account_passwords
print(apis.api_docs.show_api_doc(app_name='supervisor', api_name='show_account_passwords'))
```
Each code execution will produce an output that you can use in subsequent calls. Using these APIs, you can generate code to
↪interact with apps and solve the task.
</APPWORLD_APIS>

<APPWORLD_RULES>
Key instructions:

1. The example variables (email, passwords, etc.) are only for demonstration. Obtain the correct information by calling
↪relevant APIs.
2. Only generate valid code blocks — no ```...``` or extra formatting. Thoughts should be code comments.
3. Variables from previous code blocks persist in subsequent blocks.
4. Write small chunks of code, one chunk per step. Verify before making irreversible changes.
5. The environment only has the Python standard library. OS/filesystem modules are disabled.
6. "file system" in task instructions means the file system *app* via APIs, not the actual OS.
7. Use only provided APIs, not Python packages (e.g., do NOT `import spotify`).
8. API documentation includes input arguments and output JSON schemas. Follow them exactly.
9. For paginated APIs, make sure to consider all pages.
10. Use `datetime.now()` or the phone app for current date/time.
11. For temporal requests, use proper time boundaries (00:00:00 to 23:59:59). Single timezone.
12. References to friends/family mean people in your phone's contacts list.
13. Personal info and credentials are in the "supervisor" app.
14. Once you have completed the task, call `apis.supervisor.complete_task()`. If the task asks for information (e.g.,
↪entities, numbers) as an answer, pass it via the `answer` argument: `apis.supervisor.complete_task(answer=<answer>)`. For
↪action-only tasks (e.g., place an order, like all songs), call `apis.supervisor.complete_task()` with no arguments. The
↪agent loop ends automatically when this call returns successfully — there is no separate `finish` tool.
15. Answers should be entities or numbers, not full sentences (e.g., `answer=10` not `answer="ten"`).
16. The task is guaranteed to be completable. DO NOT END until you complete it.
17. Make all decisions autonomously — no clarifications or confirmations needed.
18. **Prefer one tool call per turn.** Emit a single tool call and wait for its result before deciding the next step. If you
↪do emit several calls in one turn they are ALL executed in order, but the later calls are written before you can see the
↪earlier calls' outputs, so they often act on stale assumptions — when in doubt, emit ONE call, read the result, then
↪continue.
</APPWORLD_RULES>
```

Figure 7: Execution core on AppWorld, the system prompt shared by both agents. Each subtask-level role appends its role block of Fig. 10, and the task-level agent uses the core unchanged.

System

```
You are an AI Assistant that completes office-productivity tasks by emitting JSON dict actions.

<AVAILABLE_APPS>
You have following apps installed in the system:
- calendar: an app to manage daily events on calendar.
- excel: an app to manipulate excel files, including reading, writing, etc.
- ocr: an app to recognize text from images.
- pdf: an app to manipulate pdf files, including format conversion and file reading.
- shell: an app to run shell commands in the system.
- word: an app to manipulate word files, including reading, writing, converting, etc.
- email: an app to manage emails, such as sending and reading emails.
- llm: an app to interact with the large language model to answer questions, generate text, etc.
</AVAILABLE_APPS>

<OFFICEBENCH_RULES>
1. You can help solve the task step by step.
2. You can interact with an operation system and use apps to solve the task.
3. You must follow the instructions and use the given json format to call APIs.
4. You can only generate one action at a time.
5. You can find files for your task in `/testbed/data`. If you don't know the filenames, please switch to shell app and call
↪commands to list the directory.
</OFFICEBENCH_RULES>
```

Figure 8: Execution core on OfficeBench, the system prompt shared by both agents. Each subtask-level role appends its role block of Fig. 11, and the task-level agent uses the core unchanged.

System

```
You are an AI Assistant that solves end-to-end data-science questions by writing Python in a persistent REPL. Each ‘run_code‘
↪ call executes Python in a PERSISTENT process — variables stay in memory across calls.

<KRAMABENCH_ENV>
- The question is answered by building a small data pipeline over real files: locate the right file(s), load, clean / type,
↪filter, transform, and compute the final value.
- The domain’s raw data lake is mounted read-only at ‘./data/‘ (your working directory is a scratch dir; ‘./data/‘ is a
↪symlink to the files). Explore it with ‘os.listdir(’data’)‘, ‘glob.glob(’data/**’, recursive=True)‘, and ‘pd.read_csv‘ / ‘
↪pd.read_excel‘.
- Available libraries: ‘pandas‘, ‘numpy‘, ‘scipy‘, plus the standard library (‘os‘, ‘glob‘, ‘json‘, ‘csv‘, ‘re‘, ‘math‘, ‘
↪pathlib‘). ‘np‘ and ‘pd‘ are pre-imported.
- The ‘run_code‘ tool takes a single ‘code‘ argument (Python source). Output (stdout + tracebacks) is returned as text,
↪truncated at ~4 KB. The kernel does NOT auto-display the last expression — use ‘print()‘.
</KRAMABENCH_ENV>

<KRAMABENCH_RULES>
Key instructions:

1. ALWAYS inspect a file before computing on it: list ‘data/‘, then ‘df.shape‘, ‘df.columns.tolist()‘, ‘df.dtypes‘, ‘df.head
↪()‘. Column / sheet-name mismatches are the #1 silent failure (xlsx files often have multiple sheets — check ‘pd.ExcelFile
↪(path).sheet_names‘).
2. Variables PERSIST across ‘run_code‘ calls — once ‘df = pd.read_csv(...)‘ is loaded it stays. Build the pipeline
↪incrementally and ‘print()‘ intermediate values to verify each step.
3. Read the question precisely: respect requested rounding, units, filters ("exclude missing values", "tumor samples only"),
↪and the exact quantity asked for. Compute the literal value — do not approximate unless asked.
4. Tracebacks come back through ‘run_code‘’s output. READ them and fix the real problem — don’t shotgun-retry the same code.
5. The question is solvable from the provided files with the available libraries. DO NOT give up; if one file/column is wrong
↪, explore others in ‘./data/‘.
6. When you have the final answer, submit it by calling ‘complete_task(answer=<value>)‘ inside ‘run_code‘. Pass the ACTUAL
↪computed value (a number, string, or list) — NOT a sentence describing it, and NOT a variable you never printed. Match the
↪ expected answer type (e.g. a single rounded float for a numeric question, a Python list for a list question). This is the
↪ SOLE completion signal — there is no separate ‘finish‘ tool, and the loop ends when this call returns.
7. Make all decisions autonomously — no clarifications or confirmations needed.
8. **Prefer one tool call per turn.** Emit a single ‘run_code‘ call, read its result, then decide the next step.
</KRAMABENCH_RULES>
```

Figure 9: Execution core on KramaBench, the system prompt shared by both agents. Each subtask-level role appends its role block of Fig. 12, and the task-level agent uses the core unchanged.

System (executor role block)

```
<ROLE>
You are executing a specific subgoal. Focus ONLY on the stated subgoal — do not work on anything else.

When (and only when) the subgoal is complete, end it by writing ‘end_subgoal()‘ inside a run_code call — there is NO separate
↪ end_subgoal tool. You may do real work and then call ‘end_subgoal()‘ in the SAME run_code block, or call it alone after
↪your last action. Always do the subgoal’s actual work FIRST — never end an empty subgoal. To finish the WHOLE task, run
↪the benchmark’s completion action (e.g. apis.supervisor.complete_task()) instead.
</ROLE>
```

System (planner role block)

```
<ROLE>
You are planning the next subgoal for a task. You see the full task and current progress.

Break the task into NO MORE THAN 5 focused subgoals. Each subgoal except the final one should not be a single API call, but a
↪ coherent step (e.g., "Login and discover relevant Spotify APIs").

CRITICAL — preserve the task’s constraints in EVERY subgoal. The executor only sees the subgoal text you write (not the
↪original task), so each subgoal must carry the task’s exact entities, filters, quantities, time windows, and conditions.
↪NEVER broaden or drop them. For example, if the task is to order *the weightlifting benches* from a cart that also holds
↪other items, the subgoal must be "remove the non-bench items, then order ONLY the weightlifting benches" — NOT "place the
↪order for all items in the cart". Likewise keep "all/only/each", specific names, counts, and date ranges verbatim in the
↪subgoal.

Always dedicate the final subgoal to completing the task via apis.supervisor.complete_task(). If the task really asks for
↪some information (e.g., entities, numbers) as answer, return it as the answer argument, i.e. call ‘apis.supervisor.
↪complete_task(answer=<answer>)‘. For tasks that only require actions and do not require an answer (e.g., like placing an
↪order), call ‘apis.supervisor.complete_task()‘.

Call plan_option(subgoal) when ready. Once the executor reports that apis.supervisor.complete_task(...) was run, the agent
↪loop ends automatically (there is no separate finish tool — keep proposing subgoals until the executor runs the completion
↪ action).
</ROLE>
```

System (summarizer role block)

```
<ROLE>
You produce a progress summary after each subgoal. This summary is the ONLY context that persists — the execution log is
↪discarded after this.

Your summary MUST include:
1. Completed steps and key outcomes
2. Environment variables: list every Python variable name that was ACTUALLY ASSIGNED in the execution log, with its type/
↪content (e.g., "Env vars: access_token (str), artists (list of dicts with ’artist_id’: int, ’name’: str)"). Only include
↪variables you can see being assigned (=) in the code. The next executor can only use variables if you mention them here.
3. APIs used: list which APIs were called (app.method), and ONLY note parameter gotchas or surprises (e.g., "login uses ’
↪username’ not ’email’", "access_token required as explicit parameter"). Do NOT try to reproduce full signatures — the next
↪ executor will read API docs directly.
4. Issues/gotchas: parameter name mismatches, pagination behavior, type constraints, failed approaches

Format: dense paragraph or compact bullet list. REPLACE the previous summary entirely — carry forward all relevant info from
↪both the old progress and the new execution.

Submit your summary via the ‘write_progress_summary‘ tool. Respond with ONLY the tool call, no other text.
</ROLE>
```

Figure 10: Role blocks of the subtask-level agent on AppWorld. Each block is appended to the execution core of Fig. 7 to form the system prompt of its role.

System (executor role block)

```
<ROLE>
You are executing a specific subgoal. Focus ONLY on the stated subgoal — do not work on anything else.

When (and only when) the subgoal is complete, end it by writing ‘end_subgoal()‘ inside a run_code call — there is NO separate
↪ end_subgoal tool. You may do real work and then call ‘end_subgoal()‘ in the SAME run_code block, or call it alone after
↪your last action. Always do the subgoal’s actual work FIRST — never end an empty subgoal. To finish the WHOLE task, run
↪the benchmark’s completion action (e.g. ‘{’app’: ’system’, ’action’: ’finish_task’, ’answer’: ...}‘) instead.
</ROLE>
```

System (planner role block)

```
<ROLE>
You are planning the next subgoal for a task. You see the full task and current progress.

Break the task into NO MORE THAN 5 focused subgoals. Each subgoal except the final one should not be a single app command (
↪JSON dict), but a coherent step (e.g., "Create the meeting on Bob’s calendar").

OfficeBench actions are concrete app commands (JSON dicts). The ONLY way to reason over content (extract / filter / compare /
↪ clean / reformat) is the ‘llm‘ app — so a step that needs reasoning is phrased as "use the llm app to ...", never as a
↪standalone non-action subgoal.

CRITICAL — preserve the task’s constraints in EVERY subgoal. The executor only sees the subgoal text you write (not the
↪original task), so each subgoal must carry the task’s exact entities, filters, quantities, time windows, and conditions.
↪NEVER broaden or drop them. For example, if the task is to schedule a meeting with *the marketing team* between 2-4pm, the
↪ subgoal must be "check Bob’s calendar and schedule the meeting with ONLY the marketing team between 2-4pm" — NOT "
↪schedule the meeting with whoever is available". Likewise keep "all/only/each", specific names, counts, and date ranges
↪verbatim in the subgoal.

Always dedicate the final subgoal to completing the task via ‘{’app’: ’system’, ’action’: ’finish_task’, ’answer’: ...}‘. If
↪the task really asks for some information (e.g., entities, numbers) as answer, return it as the answer argument, i.e. call
↪ ‘{’app’: ’system’, ’action’: ’finish_task’, ’answer’: <answer>}‘. For tasks that only require actions and do not require
↪an answer (e.g., like scheduling a meeting), call ‘{’app’: ’system’, ’action’: ’finish_task’, ’answer’: None}‘.

Call plan_option(subgoal=<the structure above>). Once the executor reports the finish_task action in the system app was
↪emitted, the loop ends automatically — keep proposing subgoals until the executor runs the completion action.
</ROLE>
```

System (summarizer role block)

```
<ROLE>
You produce a progress summary after each subgoal. This summary is the ONLY context that persists — the execution log is
↪discarded after this.

Your summary MUST include:
1. Completed steps and key outcomes
2. Discovered values: list every runtime value a later subgoal might need: file paths found, email addresses, event IDs, cell
↪ values, counts, computed results. Copy each ‘value‘ VERBATIM character-for-character (do not reformat / normalize /
↪approximate). This is the planner’s source of truth for grounding the next subgoal — be exhaustive.
3. APPs used: list which apps/actions were called, and ONLY note gotchas or surprises in their expected input or output (e.g
↪., "the llm app returns a string, so I had to json.loads it to get the dict I needed for the calendar app"). Do NOT try to
↪ reproduce full schemas — the next executor will read API docs directly.
4. Issues/gotchas: short reminders to prevent repeat errors. ONLY record a guardrail for an action that produced an EXPLICIT
↪error from the environment — NEVER second-guess an action the environment reported as successful. Cover: do-not-re-read (
↪value is already in the Discovered values above), wrong field/key corrections, and failed approaches not to retry.

Format: dense paragraph or compact bullet list. REPLACE the previous summary entirely — carry forward all relevant info from
↪both the old progress and the new execution.

Submit your summary via the ‘write_progress_summary‘ tool. Respond with ONLY the tool call, no other text.
</ROLE>
```

Figure 11: Role blocks of the subtask-level agent on OfficeBench. Each block is appended to the execution core of Fig. 8 to form the system prompt of its role.

System (executor role block)

```
<ROLE>
You are executing a specific subgoal of a data-science question. Focus ONLY on the stated subgoal — do not work on anything
↪else.

When (and only when) the subgoal is complete, end it by writing ‘end_subgoal()‘ inside a run_code call — there is NO separate
↪ end_subgoal tool. Always do the subgoal’s actual work FIRST — never end an empty subgoal. To finish the WHOLE task (only
↪on the final, completion subgoal), submit the answer with ‘complete_task(answer=<value>)‘ instead of ‘end_subgoal()‘.
</ROLE>
```

System (planner role block)

```
<ROLE>
You are planning the next subgoal for a task. You see the full task and current progress.

Break the task into NO MORE THAN 5 focused subgoals. Each subgoal except the final one should not be a single API call, but a
↪ coherent step (e.g., "Load and inspect the relevant data file").

CRITICAL — preserve the task’s constraints in EVERY subgoal. The executor only sees the subgoal text you write (not the
↪original task), so each subgoal must carry the task’s exact entities, filters, quantities, time windows, and conditions.
↪NEVER broaden or drop them. For example, if the task is to compute the average of *the ’Price’ column for all rows where ’
↪Category’ is ’Electronics’*, the subgoal must be "filter to rows where Category == ’Electronics’, then compute the mean of
↪ the Price column" — NOT "compute the mean of the Price column". Likewise keep "all/only/each", specific names, counts,
↪and date ranges verbatim in the subgoal.

Always dedicate the final subgoal to completing the task via ‘complete_task(answer=<value>)‘. If the task really asks for
↪some information (e.g., entities, numbers) as answer, return it as the answer argument, i.e. call ‘complete_task(answer=<
↪answer>)‘. For tasks that only require actions and do not require an answer (e.g., like generating a plot), call ‘
↪complete_task(answer=None)‘.

Call plan_option(subgoal) when ready. Once the executor reports that ‘complete_task(...)‘ was run, the agent loop ends
↪automatically (there is no separate finish tool — keep proposing subgoals until the executor runs the completion action).
</ROLE>
```

System (summarizer role block)

```
<ROLE>
You produce a progress summary after each subgoal. This summary is the ONLY context that persists — the execution log is
↪discarded after this.

Your summary MUST include:
1. Completed steps and key outcomes (which file(s) under ./data/ were identified and loaded, which sheet, any cleaning/
↪filtering applied)
2. Persistent Python variables still in scope with type / shape (e.g., "df (DataFrame, shape=(120, 4), columns=[...])", "
↪cohort_ids (list of str)"). The next executor can only use variables you mention here.
3. Pandas / numpy / scipy patterns + gotchas (multi-sheet xlsx, header/skiprows, dtype coercion, NaN handling, scipy result
↪attributes)
4. Any partial/intermediate numeric results already computed (copy verbatim) so the final subgoal can submit them

Format: dense paragraph or compact bullet list. REPLACE the previous summary entirely — carry forward all relevant info from
↪both the old progress and the new execution.

Submit your summary via the ‘write_progress_summary‘ tool. Respond with ONLY the tool call, no other text.
</ROLE>
```

Figure 12: Role blocks of the subtask-level agent on KramaBench. Each block is appended to the execution core of Fig. 9 to form the system prompt of its role.

User (executor)

```
{identity line}

Your current subgoal: {subgoal}

Overall task (for context only): {task instruction}

Current Progress: {progress summary}

{live action menu (OfficeBench only)}

{framed retrieved skills}

Proceed with execution. When the subgoal is complete, write ‘end_subgoal()‘ in a run_code call.
```

User (planner)

```
{identity line}

Task: {task instruction}

Current Progress: {progress summary}
```

User (summarizer)

```
Write a comprehensive progress summary that REPLACES the previous progress. Carry forward all relevant information.
```

Figure 13: User messages of the three subtask-level roles. The progress summary starts as a fixed no-progress line, and the live action menu appears only on OfficeBench.

User (retrieved text skill framing)

```
Experience notes from PREVIOUS tasks in this domain — HINTS, not ground truth. Their column / file / key names may not match
↪this task; verify against the data listing, and do not let a note change this task’s scope (filters, time range, entity,
↪units, granularity). Adapt or ignore.

{retrieved skills}
```

User (retrieved code skill framing)

```
The following Python helper functions were induced from PREVIOUS, DIFFERENT tasks. Treat them as REFERENCE, not ground truth
↪— they may NOT fit this task. Use them to recall the relevant APIs and overall approach, but write your OWN code for THIS
↪task. Do NOT blindly call a stored function assuming it works; only reuse one (by copying its definition into your code)
↪if it genuinely matches what THIS task needs, and never let a helper change this task’s goal, inputs, or scope.

{retrieved skills}
```

Figure 14: Framing wrapped around the text and code skills retrieved for the linear and subtask-level agent.

User

```
Current skill memory:
{current skill memory brief}

Look at the trajectory above — the work you JUST carried out. Write ONE reusable experience note capturing exactly that
↪procedure (no more, no less), generalized for similar future work.

<RULES>
- The ‘description‘ names the generalized procedure you just carried out (e.g. "Place an order on Amazon", NOT "Order Toshiba
↪ hard drive for Grant"); its scope = what you just did
- The FIRST bullet of ‘content‘ MUST be "Procedure: step1 → step2 → step3 → ..." summarizing exactly the procedure you
↪just carried out. Remaining bullets are individual API facts, gotchas, and patterns (unordered, appendable)
- ALWAYS use full API paths with apis. prefix: apis.app_name.api_name(param_names) — NEVER write just app_name.api_name
- Bullets should record SURPRISING or NON-OBVIOUS behaviors only: parameter name mismatches (e.g., 'username' not 'email'),
↪unexpected return key names, type constraints, edge cases. Do NOT state what an API normally does or restate its
↪documentation
- All bullets should be specific and reusable
- Do NOT include: specific emails, passwords, token values, product IDs, names
- Do NOT add a note that OVERLAPS one already in experience memory above — if a listed note already covers this procedure (
↪even under different wording), do NOT repeat it; at most add a genuinely new fact to the existing one
- If what you just did was trivially simple with no surprises or gotchas, submit the tool call with description="" and
↪content="none"
</RULES>

Example tool call (call the ‘skill_induction‘ tool with these JSON arguments):
{
  "description": "Place an order on Amazon",
  "content": "- Procedure: apis.amazon.show_cart → remove unwanted items → apis.amazon.show_payment_cards and filter
↪expired → apis.amazon.show_addresses → apis.amazon.place_order\n- apis.amazon.show_cart(access_token) → key is '
↪cart_items' NOT 'products'\n- apis.amazon.delete_product_from_cart(access_token, product_id) — NOT '
↪remove_product_from_cart'\n- apis.amazon.show_payment_cards(access_token) → 'payment_card_id' NOT 'card_id'\n- Filter
↪expired cards: expiry_year > now.year or (same year and expiry_month >= now.month)\n- apis.amazon.place_order(
↪payment_card_id, address_id, access_token) — all 3 required, return code 422 if expired\n- If insufficient balance, try
↪next valid card"
}

Respond with ONLY the function call, no other text.
```

Figure 15: Text-skill induction prompt on AppWorld, appended as a user message to the span being distilled. The same prompt serves both induction levels.

User

```
Current skill memory:
{current skill memory brief}

Look at the trajectory above — the work you JUST carried out. Write ONE reusable experience note capturing exactly that
↪procedure (no more, no less), generalized for similar future work.

<RULES>
- The ‘description‘ names the generalized procedure you just carried out (e.g. "Create a calendar event from a natural-
↪language time", NOT "Add Bob's meeting at 10:30 a.m on 5/17/2024"); its scope = what you just did
- The FIRST bullet of ‘content‘ MUST be "Procedure: step1 → step2 → step3 → ..." summarizing exactly the procedure you
↪just carried out. Remaining bullets are individual action-shape facts, gotchas, and patterns (unordered, appendable)
- ALWAYS reference actions by ‘(app, action)‘ pair with their key arguments — never write just the action name
- Bullets should record SURPRISING or NON-OBVIOUS behaviors only: key-name mismatches between sibling actions (e.g., calendar
↪ ‘create_event‘ uses ‘user‘ but ‘list_events‘ uses ‘username‘), time-format constraints, file-path conventions inside /
↪testbed, format-conversion footguns. Do NOT restate normal action behavior
- Do NOT include: specific user names, dates, file paths, or task-specific strings
- Do NOT add a note that OVERLAPS one already in experience memory above — if a listed note already covers this procedure (
↪even under different wording), do NOT repeat it; at most add a genuinely new fact to the existing one
- If what you just did was trivially simple with no surprises or gotchas, submit the tool call with description="" and
↪content="none"
</RULES>

Example tool call (call the ‘skill_induction‘ tool with these JSON arguments):
{
  "description": "Create a calendar event from a natural-language date/time",
  "content": "- Procedure: switch_app calendar → calendar.create_event with full ISO datetimes → calendar.list_events to
↪verify\n- calendar.create_event keys: user, summary, time_start, time_end (NOT username for create)\n- calendar.
↪list_events key: username (NOT user — different from create_event)\n- time_start / time_end format: 'YYYY-MM-DD HH:MM:SS'
↪(24-hour). Convert \"10:30 a.m\" → \"10:30:00\", \"5:00 p.m\" → \"17:00:00\", \"5/17/2024\" → \"2024-05-17\"\n-
↪Calendar files land at /testbed/calendar/<user>.ics — verify with shell ls /testbed/calendar"
}

Respond with ONLY the function call, no other text.
```

Figure 16: Text-skill induction prompt on OfficeBench, appended as a user message to the span being distilled. The same prompt serves both induction levels.

User

```
Current skill memory:
{current skill memory brief}

Look at the trajectory above — the work you JUST carried out. Write ONE reusable experience note capturing exactly that
↪procedure (no more, no less), generalized for similar future data-science questions over this domain's data lake.

<RULES>
- The ‘description‘ names the generalized procedure you just carried out AND the kind of data it applies to (e.g. "Locate and
↪ load the correct sheet of a proteomics xlsx", NOT "Read mmc1.xlsx in the biomedical task"); its scope = what you just did,
↪ so retrieval surfaces it for genuinely-similar steps and not for unrelated domains
- The FIRST bullet of ‘content‘ MUST be "Procedure: step1 → step2 → step3 → ..." summarizing exactly the procedure you
↪just carried out. Remaining bullets are pandas / numpy / scipy patterns, gotchas, and techniques (unordered, appendable)
- Bullets should record SURPRISING or NON-OBVIOUS behaviors only: multi-sheet xlsx (‘pd.ExcelFile(p).sheet_names‘), header/
↪skiprows quirks, dtype coercion, NaN / missing-value handling, join-key normalization, scipy result attributes (‘spearmanr
↪(...).correlation‘). Do NOT restate normal API behavior
- Data files are under ‘./data/‘; the final answer is submitted via ‘complete_task(answer=...)‘ — that is the agent's action,
↪ not a note-worthy step
- Do NOT include specific file paths, column values, or numbers from this task
- Do NOT add a note that OVERLAPS one already in experience memory above — if a listed note already covers this procedure (
↪even under different wording), do NOT repeat it; at most add a genuinely new fact to the existing one
- If what you just did was trivially simple with no surprises or gotchas, submit the tool call with description="" and
↪content="none"
</RULES>

Example tool call (call the ‘skill_induction‘ tool with these JSON arguments):
{
  "description": "Locate a file in the data lake and load the right table",
  "content": "- Procedure: ‘os.listdir('data')‘ / ‘glob.glob('data/**', recursive=True)‘ → pick the file whose name matches
↪ the question's entity → for xlsx check ‘pd.ExcelFile(<PATH>).sheet_names‘ then ‘pd.read_excel(<PATH>, sheet_name=<SHEET
↪>)‘ → inspect ‘.shape‘ / ‘.columns‘ / ‘.head()‘\n- xlsx files frequently hold several sheets; never assume the first
↪sheet is the data — list sheet names first\n- Normalize join / id columns (strip, lower, dtype) before merging or
↪filtering, or rows silently fail to match\n- Drop rows missing the relevant columns BEFORE a correlation / aggregation,
↪matching the question's 'exclude missing values' wording"
}

Respond with ONLY the function call, no other text.
```

Figure 17: Text-skill induction prompt on KramaBench, appended as a user message to the span being distilled. The same prompt serves both induction levels.

User

```
Current function memory:
{current function memory brief}

Look at the trajectory above — the work you JUST carried out. Write ONE reusable Python function capturing exactly that procedure (no more, no less), generalized for similar future work. You almost always have a procedure worth capturing — emit that one function; an empty functions list is a RARE exception.

<RULES>
- Emit EXACTLY ONE top-level function that reproduces the procedure you just carried out end-to-end. Do NOT split it into several separate top-level functions — if you need sub-steps, define them as NESTED inner functions INSIDE that one function.
- Its scope MUST MATCH what you just did: if you just carried out a whole multi-step goal, the one function performs that whole goal inline (log in, fetch, decide, act — all inside it); if you just did a single focused step, the one function is exactly that step.
- TRANSFERABLE: parametrize ALL instance-specific values (user IDs, emails, tokens, names, product IDs) as parameters. Do NOT hardcode emails, passwords, tokens, IDs, or task-specific names.
- Use full API paths with the apis. prefix: apis.app_name.api_name(param_names). Handle pagination where applicable (loop until an empty page).
- Do NOT emit a function that OVERLAPS one already in memory above — if a listed function already does this procedure (even under a different name or wording), REUSE it; never add a near-duplicate.
- Submit an EMPTY functions list ONLY in the rare case you did nothing reusable at all — i.e. the trajectory was essentially just the final apis.supervisor.complete_task() call with no real work before it. Even a single meaningful API call or lookup counts: wrap it as the one function. In every other case, emit the one function.
</RULES>

Example tool call (call the ‘skill_induction‘ tool with these JSON arguments):
{
  "functions": [
    {
      "name": "like_all_songs_from_followed_artists",
      "description": "Page through followed artists, then find and like every song by each",
      "implementation": "def like_all_songs_from_followed_artists(access_token):\n    artists = []\n    i = 0\n    while True:\n        page = apis.spotify.show_following_artists(page_index=i, access_token=access_token)\n        if not page:\n            break\n        artists.extend(page)\n        i += 1\n    liked = 0\n    for a in artists:\n        j = 0\n        while True:\n            res = apis.spotify.search_songs(page_index=j, query=a['name'], artist_id=a['id'], access_token=access_token)\n            if not res:\n                break\n            for s in res:\n                apis.spotify.like_song(access_token=access_token, song_id=s['song_id'])\n                liked += 1\n            j += 1\n    return liked"
    }
  ]
}

Respond with ONLY the function call, no other text.
```

Figure 18: Code-skill induction prompt on AppWorld, appended as a user message to the span being distilled. The same prompt serves both induction levels.

User

```
Current function memory:
{current function memory brief}

Look at the trajectory above — the work you JUST carried out. Write ONE reusable Python function capturing exactly that
↪procedure (no more, no less), generalized for similar future work. You almost always have a procedure worth capturing —
↪emit that one function; an empty functions list is a RARE exception.

<RULES>
- Emit EXACTLY ONE top-level function that reproduces the procedure you just carried out end-to-end. Do NOT split it into
↪several separate top-level functions — if you need sub-steps, define them as NESTED inner functions INSIDE that one
↪function.
- Its scope MUST MATCH what you just did: a whole multi-step goal → one function that performs the whole goal inline; a
↪single focused step → one function that does exactly that step.
- The function RETURNS the OfficeBench JSON-dict action(s) to execute — a single action dict, or a list of them in order.
↪Instance-specific values are PARAMETERS.
- Use the exact key names each app requires (e.g., calendar ‘create_event‘ uses ‘user‘, ‘list_events‘ uses ‘username‘).
- Do NOT hardcode names, dates, paths, or file contents from the current task.
- Do NOT emit a function that OVERLAPS one already in memory above — if a listed function already builds this procedure (even
↪ under a different name or wording), REUSE it; never add a near-duplicate.
- Submit an EMPTY functions list ONLY in the rare case you did nothing reusable at all — i.e. the trajectory was essentially
↪just a switch_app + finish_task with no real action before it. Even a single meaningful action counts: wrap it as the one
↪function. In every other case, emit the one function.
</RULES>

Example tool call (call the ‘skill_induction‘ tool with these JSON arguments):
{
  "functions": [
    {
      "name": "create_calendar_event_and_verify",
      "description": "Build the action sequence to create a calendar event from natural-language times, then list events to
↪verify",
      "implementation": "def create_calendar_event_and_verify(user, summary, time_start, time_end):\n    # time_start /
↪time_end must be 'YYYY-MM-DD HH:MM:SS' 24-hour strings.\n    # create_event uses key 'user'; list_events uses key '
↪username'.\n    return [\n        {'app': 'calendar', 'action': 'create_event', 'user': user, 'summary': summary, '
↪time_start': time_start, 'time_end': time_end},\n        {'app': 'calendar', 'action': 'list_events', 'username': user},\n
↪    ]"
    }
  ]
}

Respond with ONLY the function call, no other text.
```

Figure 19: Code-skill induction prompt on OfficeBench, appended as a user message to the span being distilled. The same prompt serves both induction levels.

User

```
Current function memory:
{current function memory brief}

Look at the trajectory above — the work you JUST carried out. Write ONE reusable Python function capturing exactly that
↪procedure (no more, no less), generalized for similar future data-science questions over this domain's data lake. You
↪almost always have a procedure worth capturing — emit that one function; an empty functions list is a RARE exception.

<RULES>
- Emit EXACTLY ONE top-level function that reproduces the procedure you just carried out end-to-end. Do NOT split it into
↪several separate top-level functions — if you need sub-steps (locate file, load sheet, clean, filter, merge, aggregate),
↪define them as NESTED inner functions INSIDE that one function.
- Its scope MUST MATCH what you just did: a whole multi-step analysis → one function that performs that whole analysis
↪inline; a single focused step → one function that does exactly that step.
- TRANSFERABLE: parametrize file paths, sheet / column names, group keys, filter predicates, aggregation functions. Do NOT
↪hardcode column names, file names, or values from the current task.
- Available libraries: pandas, numpy, scipy, and the standard library. Data files are under `./data/`. Do NOT call `
↪complete_task` inside the function (it is the agent's final action, not a reusable step).
- Write the `description` to name the concrete operation AND the kind of data it expects (e.g. "Spearman correlation between
↪two proteomics-abundance columns"), so retrieval matches genuinely-similar future steps and is not surfaced for unrelated
↪domains.
- Do NOT emit a function that OVERLAPS one already in memory above (even if worded differently) — REUSE it; never add a near-
↪duplicate.
- Submit an EMPTY functions list ONLY in the rare case you did nothing reusable at all — i.e. the trajectory was essentially
↪just submitting the answer via complete_task(answer=...) with no real data work before it. Even a single meaningful data
↪step counts: wrap it as the one function. In every other case, emit the one function.
</RULES>

Example tool call (call the `skill_induction` tool with these JSON arguments):
{
  "functions": [
    {
      "name": "spearman_between_columns_in_file",
      "description": "Locate a data file by keyword, load it, and compute the Spearman correlation between two named numeric
↪columns (dropping rows missing either).",
      "implementation": "def spearman_between_columns_in_file(root, file_keyword, col_a, col_b):\n    import os, pandas as pd
↪\n    from scipy.stats import spearmanr\n    def _find(root, kw):\n        for dp, _, fns in os.walk(root):\n
↪for fn in fns:\n                if kw.lower() in fn.lower():\n                    return os.path.join(dp, fn)\n
↪return None\n    path = _find(root, file_keyword)\n    df = pd.read_csv(path)\n    sub = df[[col_a, col_b]].dropna()\n
↪return spearmanr(sub[col_a], sub[col_b]).correlation"
    }
  ]
}

Respond with ONLY the function call, no other text.
```

Figure 20: Code-skill induction prompt on KramaBench, appended as a user message to the span being distilled. The same prompt serves both induction levels.

Task-Level Text Skill

```
Description: Place an order on Amazon for specific items by filtering cart contents

- Procedure: apis.supervisor.show_profile → apis.supervisor.show_account_passwords to get Amazon password → apis.amazon.
↪login(username=profile['email'], password=password) to get access_token → apis.amazon.show_cart(access_token) to retrieve
↪ cart_items → filter items by keyword → apis.amazon.clear_cart(access_token) → add each filtered item via apis.amazon.
↪add_product_to_cart → apis.amazon.show_payment_cards(access_token) and filter by expiry → select first valid
↪payment_card_id → apis.amazon.show_addresses(access_token) → select address_id → apis.amazon.place_order(access_token,
↪payment_card_id, address_id) → apis.supervisor.complete_task()
- Amazon login does NOT use stored tokens between sessions — must re-login on each task
- apis.amazon.show_cart(access_token) returns full cart with 'cart_items' as list of product dicts
- Payment and address IDs come from Amazon's apis.amazon.show_payment_cards and apis.amazon.show_addresses, NOT from
↪supervisor app
- Card expiry check: filter cards where (expiry_year > current_year) or (same year and expiry_month >= current_month)
- Order placement fails with 422 if cart is empty or missing payment_card_id / address_id
- To filter cart by category, must clear and re-add only matching items since no bulk remove by filter API exists
- apis.amazon.login requires 'username' field, not 'email' — unexpected field name
- acces_token from login is a dict with 'access_token' string inside
- apis.amazon.show_cart returns full cart with cost breakdown and 'cart_items' list
- No API to remove items by filter; must clear and re-add
- apis.amazon.clear_cart(access_token) removes all items
- apis.amazon.add_product_to_cart requires product_id and quantity
- Payment card expiry must be checked using current year/month; expired cards cause 422 on order
- apis.amazon.place_order requires access_token, payment_card_id, and address_id — all three
```

Figure 21: The task-level text skill induced from the example task. One workflow note spans the whole trajectory from login to order placement.

Task-Level Code Skill

```
Description: Filter cart for weightlifting benches, then place order using valid payment and shipping

def place_weightlifting_benches_order(access_token, product_keywords=['weightlifting bench']):
    # Retrieve full cart
    cart = apis.amazon.show_cart(access_token=access_token)

    # Filter for relevant product types
    filtered_items = [item for item in cart['cart_items'] if any(kw in item['product_name'].lower() for kw in
↪product_keywords)]

    if not filtered_items:
        raise Exception("No weightlifting benches found in cart")

    # Clear cart and re-add only filtered items
    apis.amazon.clear_cart(access_token=access_token)
    for item in filtered_items:
        apis.amazon.add_product_to_cart(
            product_id=item['product_id'],
            quantity=item['quantity'],
            access_token=access_token
        )

    # Get valid payment method (not expired)
    from datetime import datetime
    now = datetime.now()
    cards = apis.amazon.show_payment_cards(access_token=access_token)
    valid_card = next(
        c for c in cards
        if c['expiry_year'] > now.year or (c['expiry_year'] == now.year and c['expiry_month'] >= now.month)
    )

    # Get shipping address
    addresses = apis.amazon.show_addresses(access_token=access_token)
    if not addresses:
        raise Exception("No shipping address found")

    # Place final order
    return apis.amazon.place_order(
        access_token=access_token,
        payment_card_id=valid_card['payment_card_id'],
        address_id=addresses[0]['address_id']
    )
```

Figure 22: The task-level code skill induced from the same task. One function reproduces the whole trajectory and keeps the source task's product keyword as its default argument.

Subtask-Level Text Skills

```
Description: Login to an app using supervisor-provided credentials

- Procedure: apis.supervisor.show_account_passwords() → filter by 'account_name' → apis.<app>.login(username, password) →
↪ extract 'access_token'
- apis.<app>.login() param is 'username', not 'email' — always verify via show_api_doc
- Access token returned in 'access_token' key of response
- Store access_token in variable for later API calls
- Amazon login uses 'username' (accepts email address), not 'email'
- Passwords may contain special characters — use as-is without modification
- 'access_token' from login is required for subsequent API calls and may expire
- Always confirm login parameter names using apis.api_docs.show_api_doc


Description: Filter and retain specific items in a cart by product name

- Procedure: apis.amazon.show_cart(access_token) → examine returned structure (key 'cart_items') → iterate over items →
↪conditionally call apis.amazon.delete_product_from_cart(access_token, product_id) for non-matching items
- apis.amazon.show_cart(access_token) returns a dict with metadata (total_cost, tax) and 'cart_items' as a list of dicts
- 'product_id' is integer, not string — pass as int to delete_product_from_cart
- apis.amazon.delete_product_from_cart requires both product_id and access_token
- Always re-check cart contents after deletions using show_cart to confirm success
- apis.amazon.show_cart(access_token) returns cart with key 'cart_items', not 'items'
- Removal is done via apis.amazon.delete_product_from_cart, not 'remove_from_cart'
- Always verify cart structure before iterating to avoid KeyError on 'items'
- Use product_id as integer, not string, in calls


Description: Complete a task by placing an order for items in Amazon cart

- Procedure: call apis.supervisor.complete_task() with no arguments
- apis.supervisor.complete_task() should be called with no arguments when the task is action-based (e.g. place an order, like
↪ songs)
- Calling apis.supervisor.complete_task() ends the agent loop and completes the task
```

Figure 23: The subtask-level text skills induced from the same task, one note per completed subtask.

Subtask-Level Code Skills

```
Description: Login to Amazon and retrieve the shopping cart contents

def login_and_get_cart(email, password):
    login_result = apis.amazon.login(username=email, password=password)
    access_token = login_result['access_token']
    cart = apis.amazon.show_cart(access_token=access_token)
    return cart


Description: Remove items from Amazon cart that do not match a keyword in their product name

def clean_cart_of_non_matching_items(access_token, keyword):
    # Retrieve the current cart
    full_cart = apis.amazon.show_cart(access_token=access_token)
    cart_items = full_cart['cart_items']

    # Identify items that do not contain the keyword in their name
    non_matching_items = [item for item in cart_items if keyword.lower() not in item['product_name'].lower()]

    # Remove each non-matching item
    for item in non_matching_items:
        apis.amazon.delete_product_from_cart(access_token=access_token, product_id=item['product_id'])

    # Return the cleaned list of remaining items
    updated_cart = apis.amazon.show_cart(access_token=access_token)
    return updated_cart['cart_items']


Description: Place an Amazon order using the first valid (non-expired) payment card and specified address

def place_order_with_valid_payment(access_token, address_id):
    from datetime import datetime
    cards = apis.amazon.show_payment_cards(access_token=access_token)
    now = datetime.now()
    current_year, current_month = now.year, now.month

    valid_card = None
    for card in cards:
        exp_year = card['expiry_year']
        exp_month = card['expiry_month']
        if exp_year > current_year or (exp_year == current_year and exp_month >= current_month):
            valid_card = card
            break

    if valid_card is None:
        raise Exception('No valid payment card found')

    payment_card_id = valid_card['payment_card_id']
    return apis.amazon.place_order(access_token=access_token, payment_card_id=payment_card_id, address_id=address_id)
```

Figure 24: The subtask-level code skills induced from the same task, one function per completed subtask.

E Additional Analyses

This appendix provides supporting analyses for the main results. Fig. 26 shows that the two headline comparisons of Sec. 5 survive weaker induction prompts, and Fig. 27 shows the specificity-abstractness tradeoff behind the skill utility score. All figures use the same common task subset as the main results.

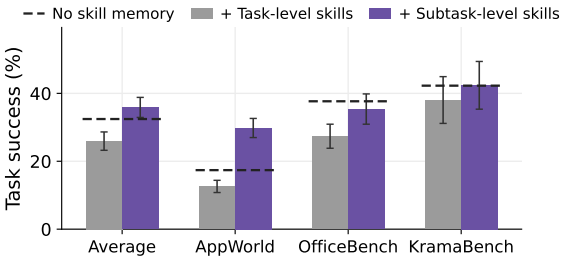

Figure 25: Task success of the task-level agent retrieving its original task-level skills or the subtask-level agent's skills, averaged over the Text and Code formats and pooled over Qwen3-235B-A22B and GPT-OSS-120B, with 95% task-bootstrap confidence intervals. Dashes mark the task-level agent without skill memory. With its original skills the task-level agent falls below the dashes on every benchmark, while with subtask-level skills it beats the original skills everywhere and returns to or above the dashes.

E.1 Full Results of All Models

Tab. 3 extends Tab. 2 with the two weakest models, Gemma-3-4B and Gemma-3-12B. Gemma-3-4B stays at or near the floor on all three benchmarks, and Gemma-3-12B stays at the floor on AppWorld. Every Average row and every number quoted in the main text already includes both models.

E.2 Task-Level Agent with Subtask Skills

We verify that the gap between the two induction levels comes from the induced skills rather than from the agent that uses them. We replay the skill memory induced by the subtask-level agent into the task-level agent at retrieval time, so the task-level agent enters each task with the same library the subtask-level agent had at that point, and we freeze induction so the library stays the subtask-level agent's own.

Fig. 25 compares the task-level agent retrieving subtask skills against the same agent retrieving its original task-level skills, averaged over the two skill formats, on Qwen3-235B-A22B and GPT-OSS-120B. Subtask-level skills beat the original skills on every benchmark, by 9.9 points on average and up to 17.2 points on AppWorld. The original skills drag the task-level agent below its

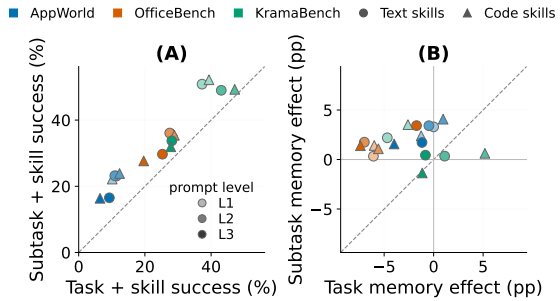

Figure 26: Subtask-level induction stays ahead of task-level induction at every induction-prompt level. Each point is one benchmark, skill format, and prompt level, where L3 is the deployed prompt and L1 and L2 are the reduced versions of Sec. 4, averaged over the models that ran it. Panel (A) compares the task success of the two skill arms, and panel (B) compares each arm's memory effect against its own no-memory baseline on the same tasks. Points above the dashed diagonal favor the subtask level, and the upper left quadrant of (B) holds the points where task-level memory hurts while subtask-level memory helps.

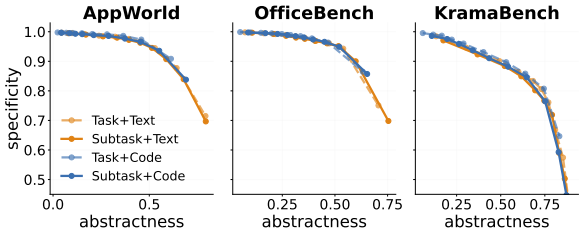

Figure 27: The tradeoff between the two terms of the skill utility score. Each panel bins the skills of one benchmark into equal-size abstractness deciles and plots mean specificity against mean abstractness for the four skill conditions. All conditions fall on nearly one frontier, and subtask induction trades a small loss in specificity for a large gain in abstractness, which raises the product.

no-memory baseline on every benchmark, while the subtask skills return it to the level of that baseline on OfficeBench and KramaBench and lift it 12.4 points above on AppWorld. The same agent thus turns from harmed to helped once its memory holds subtask-level skills, so the effect of the induction level travels with the skills.

E.3 Outcomes of Skill Source Tasks

Skill induction is not gated on task outcome in either agent. For each induced skill we therefore check whether the task it was induced from was solved in that run. Pooled over all models and benchmarks, the share of skills induced from unsolved tasks is 74.7% for Task+Text, 83.0% for Task+Code, 75.1% for Subtask+Text, and 75.8% for Subtask+Code. The two levels draw on source tasks of similar outcomes under both formats, so the opposite skill effects in Tab. 2 are not explained

Benchmark	Model	Task-level			Subtask-level		
		None	+Text	+Code	None	+Text	+Code
AppWorld	Qwen3-235B-A22B	7.4 [5.0, 10.1]	18.0 [14.4, 21.8]	14.4 [11.0, 18.0]	27.3 [23.0, 31.7]	35.5 [30.9, 40.0]	**35.7** [31.2, 40.3]
	GPT-OSS-120B	**27.3** [23.3, 31.7]	17.0 [13.4, 20.6]	1.0 [0.2, 1.9]	26.4 [22.3, 30.5]	25.2 [21.1, 29.5]	23.7 [19.7, 27.8]
	Nemotron-Super-120B	8.9 [6.2, 11.8]	4.3 [2.4, 6.2]	1.4 [0.5, 2.6]	17.7 [14.1, 21.6]	**21.6** [17.7, 25.4]	18.5 [14.9, 22.3]
	Qwen3-4B	0.0 [0.0, 0.0]	0.2 [0.0, 0.7]	0.0 [0.0, 0.0]	0.2 [0.0, 0.7]	0.7 [0.0, 1.7]	**1.4** [0.5, 2.6]
	Qwen3-8B	0.2 [0.0, 0.7]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	2.2 [1.0, 3.6]	**3.1** [1.7, 5.0]	1.9 [0.7, 3.4]
	Qwen3-14B	0.5 [0.0, 1.2]	0.7 [0.0, 1.7]	0.2 [0.0, 0.7]	6.7 [4.3, 9.4]	7.2 [4.8, 9.8]	**8.4** [5.8, 11.0]
	Qwen3-32B	1.9 [0.7, 3.4]	2.9 [1.4, 4.6]	1.4 [0.5, 2.6]	7.7 [5.3, 10.3]	**10.3** [7.4, 13.2]	7.2 [4.8, 9.8]
	Gemma-3-4B	**0.0** [0.0, 0.0]	**0.0** [0.0, 0.0]	**0.0** [0.0, 0.0]	**0.0** [0.0, 0.0]	**0.0** [0.0, 0.0]	**0.0** [0.0, 0.0]
	Gemma-3-12B	0.0 [0.0, 0.0]	0.2 [0.0, 0.7]	0.2 [0.0, 0.7]	**1.7** [0.5, 3.1]	1.4 [0.5, 2.6]	1.0 [0.2, 1.9]
	Gemma-3-27B	0.5 [0.0, 1.2]	0.5 [0.0, 1.2]	0.0 [0.0, 0.0]	4.6 [2.6, 6.7]	4.3 [2.4, 6.5]	**4.8** [2.9, 7.0]
	Gemini-3.1-Pro	68.1 [63.5, 72.4]	58.0 [53.2, 62.6]	52.3 [47.5, 57.1]	68.3 [63.8, 72.7]	72.4 [68.1, 76.7]	**77.5** [73.4, 81.5]
	Average	10.4 [9.6, 11.3]	9.3 [8.4, 10.2]	6.5 [5.8, 7.1]	14.8 [13.5, 16.2]	**16.5** [15.1, 18.0]	16.4 [15.1, 17.7]
OfficeBench	Qwen3-235B-A22B	**43.0** [37.3, 48.7]	38.3 [33.0, 44.0]	36.7 [31.3, 42.0]	38.7 [33.3, 44.3]	41.3 [35.7, 47.0]	38.7 [33.3, 44.0]
	GPT-OSS-120B	32.3 [27.0, 37.7]	29.3 [24.3, 34.3]	5.0 [2.7, 7.7]	38.0 [32.7, 43.3]	**42.0** [36.7, 47.7]	40.7 [35.3, 46.3]
	Nemotron-Super-120B	28.3 [23.3, 33.7]	31.7 [26.3, 37.0]	12.7 [9.0, 16.7]	27.3 [22.3, 32.3]	**37.7** [32.3, 43.3]	25.0 [20.0, 30.0]
	Qwen3-4B	17.7 [13.3, 22.0]	16.3 [12.3, 20.7]	13.0 [9.3, 17.0]	18.0 [13.7, 22.7]	**25.3** [20.7, 30.3]	20.3 [16.0, 25.0]
	Qwen3-8B	**25.0** [20.0, 30.0]	15.7 [11.7, 20.0]	16.3 [12.3, 20.7]	19.0 [14.7, 23.7]	**25.0** [20.3, 30.0]	24.7 [19.7, 29.7]
	Qwen3-14B	28.3 [23.3, 33.7]	24.0 [19.3, 29.0]	27.3 [22.3, 32.3]	27.0 [22.0, 32.0]	**32.0** [27.0, 37.3]	30.3 [25.3, 35.7]
	Qwen3-32B	34.3 [29.0, 39.7]	32.7 [27.3, 38.0]	35.3 [30.0, 41.0]	33.3 [28.3, 38.7]	**36.0** [30.7, 41.7]	31.7 [26.7, 37.0]
	Gemma-3-4B	**3.7** [1.7, 6.0]	3.0 [1.3, 5.0]	2.3 [0.7, 4.3]	2.7 [1.0, 4.7]	1.0 [0.0, 2.3]	0.7 [0.0, 1.7]
	Gemma-3-12B	10.0 [6.7, 13.7]	19.7 [15.3, 24.3]	11.0 [7.7, 14.7]	13.7 [10.0, 17.7]	**21.3** [17.0, 26.0]	20.0 [15.7, 24.7]
	Gemma-3-27B	**28.3** [23.3, 33.7]	26.7 [21.7, 31.7]	14.7 [10.7, 18.7]	24.0 [19.0, 29.0]	19.0 [14.7, 23.7]	23.7 [19.0, 28.7]
	Gemini-3.1-Pro	46.3 [40.7, 52.0]	41.0 [35.3, 46.7]	41.7 [36.3, 47.3]	47.3 [41.7, 53.0]	46.0 [40.3, 51.7]	**48.7** [43.0, 54.3]
	Average	27.0 [23.6, 30.4]	25.3 [22.1, 28.8]	19.6 [17.0, 22.4]	26.3 [23.0, 29.7]	**29.7** [26.2, 33.2]	27.7 [24.2, 31.2]
KramaBench	Qwen3-235B-A22B	52.8 [43.0, 62.5]	49.3 [39.2, 59.2]	51.5 [41.1, 61.2]	52.4 [42.5, 62.2]	**55.3** [45.4, 64.7]	52.7 [42.9, 62.7]
	GPT-OSS-120B	31.7 [22.8, 41.1]	28.4 [19.6, 37.4]	22.5 [14.2, 31.1]	48.1 [38.3, 58.2]	49.8 [39.6, 59.5]	**50.9** [40.8, 60.6]
	Nemotron-Super-120B	53.3 [43.2, 63.2]	52.1 [42.1, 62.2]	43.3 [33.5, 53.4]	**59.8** [50.0, 69.1]	57.6 [47.6, 67.6]	49.9 [40.1, 59.7]
	Qwen3-4B	8.3 [3.3, 14.4]	12.3 [6.2, 19.0]	13.3 [6.9, 20.4]	13.8 [7.5, 20.8]	**15.1** [8.3, 22.5]	11.5 [5.5, 18.1]
	Qwen3-8B	10.7 [4.8, 17.2]	**15.5** [8.7, 22.9]	14.9 [8.3, 22.0]	14.4 [7.7, 21.5]	11.2 [5.4, 17.7]	14.1 [7.5, 21.5]
	Qwen3-14B	20.1 [12.5, 28.2]	14.9 [8.5, 22.0]	23.1 [14.7, 31.6]	19.1 [11.5, 27.1]	**23.5** [15.2, 32.5]	20.3 [12.5, 28.6]
	Qwen3-32B	30.0 [21.1, 39.2]	25.8 [17.6, 34.8]	30.2 [21.4, 39.3]	34.3 [25.0, 43.9]	**39.0** [29.3, 48.6]	38.9 [29.2, 48.7]
	Gemma-3-4B	1.5 [0.0, 4.0]	1.1 [0.0, 3.3]	1.9 [0.0, 4.9]	**6.0** [1.6, 11.4]	4.3 [1.0, 8.6]	2.2 [0.0, 5.4]
	Gemma-3-12B	16.3 [9.7, 23.7]	14.0 [7.3, 21.2]	11.8 [6.0, 18.2]	**17.9** [10.7, 25.7]	17.7 [10.6, 25.3]	14.2 [7.9, 21.3]
	Gemma-3-27B	19.7 [12.2, 27.7]	22.0 [14.0, 30.6]	18.6 [11.3, 26.5]	**25.3** [16.7, 34.4]	23.9 [15.9, 32.7]	24.2 [16.0, 33.1]
	Gemini-3.1-Pro	74.3 [65.6, 82.5]	74.1 [65.3, 82.4]	75.1 [66.4, 83.3]	**75.2** [66.5, 83.2]	73.7 [64.9, 82.0]	72.4 [63.6, 80.8]
	Average	29.0 [24.1, 34.0]	28.1 [23.2, 33.3]	27.8 [23.0, 32.9]	33.3 [28.0, 38.7]	**33.7** [28.5, 39.1]	31.9 [26.8, 37.1]
Average (11 models)		22.1 [20.2, 24.2]	20.9 [18.9, 22.9]	18.0 [16.1, 19.9]	24.8 [22.7, 27.0]	**26.7** [24.5, 28.8]	25.3 [23.3, 27.4]

Table 3: Task success (%) of all eleven models, the full version of Tab. 2 with Gemma-3-4B and Gemma-3-12B included. Brackets are 95% task-bootstrap confidence intervals. Each in-block Average row is taken over the models on that benchmark, and the bottom row over all models and benchmarks.

by one level distilling more failed experience than the other.

E.4 Causal Effect of Skill Utility

We test the causal effect of skill utility by intervening on the skill library while holding everything else fixed. We split each library at its median skill utility into two complementary halves and rerun the task-level agent on the same 92 KramaBench tasks with only one half in memory, using the per-task replay of App. E.2 with induction frozen, for both the task-level and the subtask-level library. Retrieval is plain top-k, so the two arms run the same tasks and receive the same number of skills per task, and differ only in which skills they receive.

Tab. 4 shows the result, averaged over the Text and Code formats and pooled over Qwen3-235B-A22B and GPT-OSS-120B. The high-utility half outperforms the low-utility half for both the task-level library, with 44.0% against 42.9%, and the subtask-level library, with 48.4% against 47.0%. The score, computed from skill descriptions and task instructions alone, thus identifies in advance the half of a library that produces more successes on the same tasks.

Skill library	High-utility half	Low-utility half
Task-level	**44.0** [36.7, 51.4]	42.9 [35.6, 50.0]
Subtask-level	**48.4** [41.0, 55.7]	47.0 [39.1, 54.9]

Table 4: Task success (%) of the task-level agent retrieving only the high-utility or only the low-utility half of the same skill library, split at the median skill utility, on KramaBench, averaged over the Text and Code formats and pooled over Qwen3-235B-A22B and GPT-OSS-120B, with 95% task-bootstrap confidence intervals. Both arms run the same tasks with the same number of retrieved skills per task, and the high-utility half wins for both libraries.

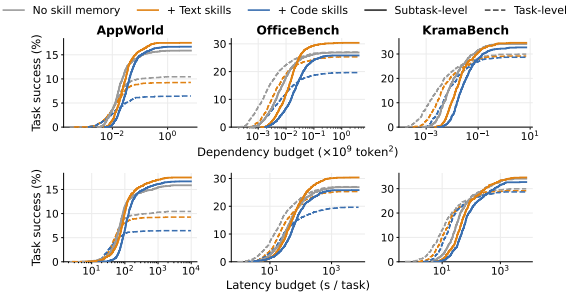

Figure 28: Task success reachable within a per-task budget of dependency (left column) and latency (right column), one row per benchmark, pooled over models, with color the skill format and line style the induction level as in Fig. 3. The crossover of the main text appears on every benchmark, where the task-level agent leads at small budgets and the subtask-level agent overtakes and saturates higher.

Benchmark	Difficulty	Task score	Retrieved utility
AppWorld	Level 1	0.215	0.250
	Level 2	0.115	0.258
	Level 3	0.082	0.257
OfficeBench	1 app	0.320	0.294
	2 apps	0.353	0.286
	3 apps	0.107	0.277
KramaBench	Easy	0.451	0.362
	Hard	0.240	0.321

Table 5: Mean task score and mean utility of the retrieved skills at each benchmark's native difficulty levels, pooled over all models, both skill formats, and both agents. The task score varies severalfold across the levels while the utility of retrieved skills stays nearly constant.

The score is also stable across task conditions. Across each benchmark's native difficulty levels, the mean utility of retrieved skills stays nearly constant, moving only from 0.250 to 0.257 on AppWorld, from 0.294 to 0.277 on OfficeBench, and from 0.362 to 0.321 on KramaBench, pooled over all models, both skill formats, and both agents (Tab. 5). Within any fixed difficulty level, higher retrieved utility still accompanies higher success, with Spearman $\rho = +0.095$ for the task-level agent and $\rho = +0.075$ for the subtask-level agent, both at $p < 10^{-10}$.

E.5 Reuse Frequency and Skill Format

Fig. 6 pools the two skill formats on purpose, because reuse frequency tracks the induction-level effect but not the format effect. Code skills are retrieved as often as Text skills, or more often, on five of the six benchmark and format combinations, yet they bring smaller gains (Sec. 5.2). Reuse frequency measures how often a skill is retrieved, not how broadly it applies, so it separates the two induction levels but not the two formats. We therefore rank skill memories with the skill utility score, which reads the format effect off abstractness, rather than with raw reuse counts.

Induction level	+Text	+Code
Task-level	75.6 [72.8, 78.3]	88.1 [86.2, 89.9]
Subtask-level	79.6 [78.1, 80.9]	86.1 [85.0, 87.2]

Table 6: Self-retrieval rates on Qwen3-235B-A22B, Nemotron-Super-120B, and Qwen3-32B, pooled over the three benchmarks, with 95% task-bootstrap confidence intervals. A skill counts as *self-retrievable* if it ranks among the original skills retrieved for its source task or subtask. All conditions use the same retrieval configuration.

E.6 Validation of Retrieval Quality

We inspect the retriever to validate that the skill effect does not stem from retriever quality. We hypothesize that a skill induced from a task should be retrievable from that same task. Specifically, we compute the cosine similarity between each skill and the task or subtask from which it was induced. A skill counts as *self-retrievable* if this similarity exceeds the minimum similarity between that task or subtask and its originally retrieved skills. In other words, the skill would rank inside the original retrieval set of its source. All conditions use the deployed embedder and the same retrieval thresholds, so both formats and both induction levels face the same bar.

Tab. 6 reports self-retrieval rates on Qwen3-235B-A22B, Nemotron-Super-120B, and Qwen3-32B, pooled over the three benchmarks. First, the two induction levels are statistically indistinguishable, so retrieval quality cannot explain the level effect. Second, Code skills self-retrieve better than Text skills, yet Sec. 5.2 shows that Text skills help more than Code skills, so retrieval quality cannot explain the format effect either. We hypothesize that the low self-retrieval rate of Text skills arises because Text descriptions are the most abstract. On KramaBench, nearly half of the task-level Text skills are not self-retrievable because their descriptions keep the reusable wrangling pattern but drop the topical words of the question. In Sec. 6.2, however, we show that this same abstraction leads to better cross-task skill transfer.

As an additional post-hoc check, we sample 50 induced skills per condition from the same population and ask Gemini-3.1-Pro whether each skill is relevant to its source task. The judge marks 100, 100, 98, and 100 percent of the skills as relevant

Artifacts/Models/Packages	Citation	Link	License
		Benchmarks	
AppWorld	Trivedi et al. (2024)	https://github.com/StonyBrookNLP/appworld	Apache-2.0 License
OfficeBench	Wang et al. (2024c)	https://github.com/zlwang-cs/OfficeBench	Apache-2.0 License
KramaBench	Lai et al. (2026)	https://github.com/mitdbg/Kramabench	MIT License (Code) and CC-BY-NC-4.0 (data)
		Backbone Models	
Qwen3 (4B, 8B, 14B, 32B, 235B-A22B)	Yang et al. (2025)	https://huggingface.co/collections/Qwen/qwen3	Apache-2.0 License
GPT-OSS-120B	Agarwal et al. (2025)	https://huggingface.co/openai/gpt-oss-120b	Apache-2.0 License
Nemotron-3-Super-120B	Chandiramani et al. (2026)	https://huggingface.co/nvidia/NVIDIA-Nemotron-3-Super-120B-A12B-BF16	NVIDIA Open Model License
Gemma-3 (4B, 12B, 27B)	Team et al. (2025)	https://deepmind.google/models/gemma/gemma-3/	Gemma Terms of Use
Gemini-3.1-Pro	N/A	https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview	Missing
all-MiniLM-L6-v2	Reimers and Gurevych (2019)	https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2	Apache-2.0 License
		Packages	
vLLM	Kwon et al. (2023)	https://github.com/vllm-project/vllm	Apache-2.0 License
boto3	N/A	https://github.com/boto/boto3	Apache-2.0 License
google-genai	N/A	https://github.com/googleapis/python-genai	Apache-2.0 License
Sentence-Transformers	Reimers and Gurevych (2019)	https://github.com/huggingface/sentence-transformers	Apache-2.0 License
NumPy	Harris et al. (2020)	https://numpy.org	BSD 3-Clause License
pandas	Wes McKinney (2010)	https://pandas.pydata.org	BSD 3-Clause License
SciPy	Virtanen et al. (2020)	https://scipy.org	BSD 3-Clause License
Matplotlib	Hunter (2007)	https://matplotlib.org	Matplotlib License (PSF-based)
Tesseract OCR	Kay (2007)	https://github.com/tesseract-ocr/tesseract	Apache-2.0 License
LibreOffice	N/A	https://www.libreoffice.org	Mozilla Public License 2.0

Table 7: Benchmarks, backbone models, and major software packages used in our study, with their citations, links, and licenses. We will release the codebase and the induced skill libraries of this project under the MIT License to support open science and reproducibility.

for Task+Text, Task+Code, Subtask+Text, and Subtask+Code, respectively. Induced skills are thus almost always relevant to their source task in every condition, so differences in skill relevance cannot explain the skill effect.

F Responsible NLP Research

F.1 Artifacts

To foster reproducibility and open science, we will release our complete codebase and the induced skill libraries of all conditions under the MIT License. Tab. 7 details the essential artifacts of this project, covering the benchmarks, the backbone models, and the major software packages. We adhere to the intended use of every artifact, and each license permits non-commercial research applications.

All benchmark tasks and all induced skills are in English. The released artifacts contain no personally identifiable information, because AppWorld and OfficeBench run on fully simulated user data and KramaBench draws on public scientific files. For the same reason the artifacts contain no offensive content.

F.2 Usage of AI Assistants

We use Artificial Intelligence (AI) assistants in three roles, a judge inside one validation experiment, coding support, and writing support.

AI Judge. The retrieval-quality check of App. E.6 uses Gemini-3.1-Pro to judge whether sampled induced skills are relevant to their source tasks. This judge is part of the reported experiments rather than of the paper production, and its setup and results appear there.

AI Code Completions. To streamline the development process, we leveraged Claude Code for assistance. The tool was primarily used to generate routine code, such as inline comments, function header documentation, and boilerplate statements like `if __name__ == "__main__":`. The high-level software architecture and the core logic of all functions were manually designed and implemented by the authors.

Grammar Checking. The initial draft of this paper was composed manually. For refinement, we employed a suite of AI-powered writing assistants, including DeepL for translation, and Claude Code, Gemini for grammatical correctness.